Tools of Artificial Intelligence

Classification Module

Smith K. Khare

The Mærsk Mc Kinney Møller Institute
University of Southern Denmark

February 16, 2026

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Outline

Classification pipeline

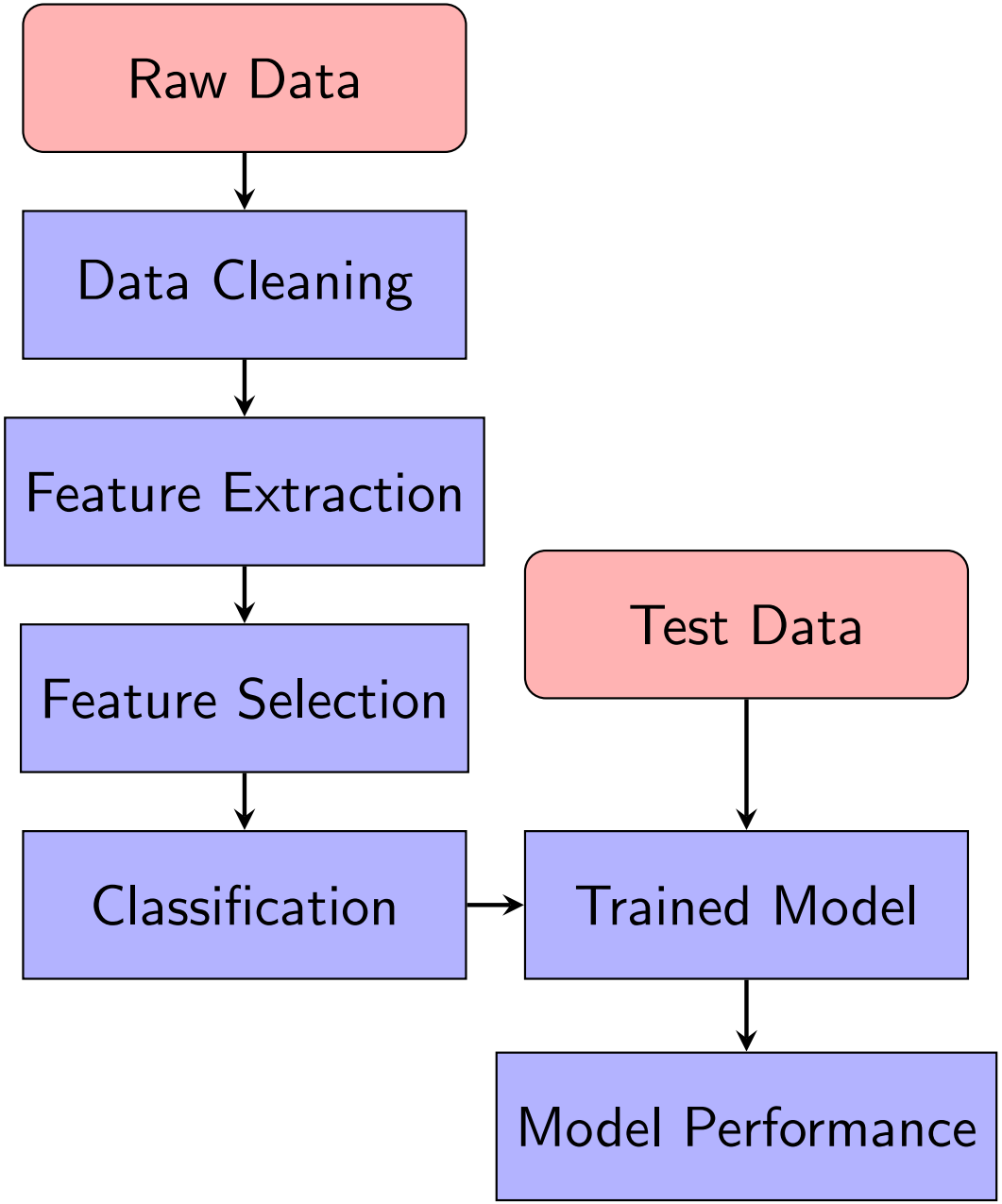
Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Machine Learning (Classification) Pipeline



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Importance of Data Cleaning and Preprocessing

- **Improves Data Quality**: Ensures the dataset is accurate, consistent, and usable.
- **Prevents Bias**: Removes or corrects errors and inconsistencies that could lead to misleading results.
- **Enhances Model Performance**: Clean data improves the performance of machine learning models by providing them with high-quality inputs.
- **Saves Resources**: Properly preprocessed data ensures that time and resources are not wasted on irrelevant or noisy data.
- **Reduces Overfitting**: Proper preprocessing techniques help to avoid overfitting by removing irrelevant features or noisy data points.
- **Improves Generalization**: Well-prepared data enables the model to generalize better to unseen data.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Techniques for Data Cleaning and Preprocessing

- Handling Missing Data:
  - Imputation (mean, median, mode, or advanced methods like k-NN).
  - Deleting rows or columns with excessive missing values.
  - Using interpolation for time series data.
- Handling Outliers:
  - Identifying outliers using statistical tests or visualization (e.g., box plots).
  - Removing or correcting outliers based on domain knowledge.
- Data Transformation:
  - Scaling (e.g., min-max scaling, z-score normalization).
  - Encoding categorical variables (e.g., one-hot encoding, label encoding).
- Feature Engineering:
  - Creating new features from existing data to better represent patterns.
  - Feature selection and extraction techniques.
- Filtering:
  - Using filters to remove noise or irrelevant data from the dataset.
  - Techniques like moving averages, Gaussian filters, or Fourier transforms for denoising or smoothing.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Time Series Data Preprocessing Techniques

- Handling Missing Data:
  - Interpolation (linear, cubic, etc.).
  - Forward or backward filling.
- Trend and Seasonality Decomposition:
  - Use techniques like STL (Seasonal and Trend decomposition using Loess) to remove trends and seasonality.
- Scaling and Normalization:
  - Min-max scaling or z-score normalization of time series data to ensure uniformity.
- Filtering:
  - Apply filters to remove high-frequency noise in time series data.
  - Common filters include Gaussian smoothing, Kalman filters, and moving averages.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Image Data Preprocessing Techniques

- Resizing:
  - Standardize the size of images to ensure consistency for model input.
- Normalization:
  - Scale pixel values to a range (e.g., [0, 1] or [-1, 1]).
- Data Augmentation:
  - Techniques like rotation, flipping, cropping, and scaling to generate more training data.
- Noise Removal:
  - Apply filters (e.g., Gaussian blur) to remove noise from images.
- Color Space Conversion:
  - Convert to different color spaces (e.g., RGB to grayscale or HSV) to improve model performance.
- Edge Detection:
  - Use techniques like Sobel or Canny edge detection to highlight significant features.
- Filtering:
  - Apply filters like Gaussian blur, median filter, or bilateral filters to smooth the image and reduce noise.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Challenges in Data Cleaning and Preprocessing

- **Handling Imbalanced Data:** In many cases, data may be imbalanced, which can affect the performance of the model.
- **Noise in Data**: Irrelevant or incorrect data points can negatively affect the performance of machine learning algorithms.
- **Data Leakage**: Leakage of information from outside the training dataset can occur, which leads to overly optimistic model performance.
- **Computational Complexity**: Some preprocessing techniques, especially in large datasets, may be computationally expensive and time-consuming.
- **Preserving Data Distribution**: Preprocessing should not distort the original data distribution, especially in imbalanced classes.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Preprocessing using filtering

```python
from scipy.signal import butter, filtfilt, iirnotch

def bandpass_filter(data, lowcut, highcut, fs, order=7):
        nyquist = 0.5 * fs
        low = lowcut / nyquist
        high = highcut / nyquist
        b, a = butter(order, [low, high], btype='band')
        y = filtfilt(b, a, data)
        return y

def notch_filter(data, notch_freq, fs, quality_factor=20):
        nyquist = 0.5 * fs
        notch = notch_freq / nyquist
        b, a = iirnotch(notch, quality_factor)
        y = filtfilt(b, a, data)
        return y

fs = 100  # sampling frequency in Hz
# Apply notch filter to suppress 50 Hz signal
filtered_RF = notch_filter(RF, 50, fs)
filtered_RH = notch_filter(RH, 50, fs)

# Apply bandpass filter between 0.5-30 Hz
filtered_signal_RF = bandpass_filter(filtered_RF, 0.5, 30, fs)
filtered_signal_RH = bandpass_filter(filtered_RH, 0.5, 30, fs)
```


Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Importance of Features in Machine Learning

- Features are the input variables used by machine learning models to make predictions.
- Good features can significantly improve the performance of a model.
- Feature extraction helps in reducing dimensionality and improving computational efficiency.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Definition of Features

- A feature is an individual measurable property or characteristic of a phenomenon being observed.
- Features are also known as attributes, variables, or predictors.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Types of Features

- Statistical Features
- Texture Features
- Entropy Features
- Image Features
- Frequency Domain Features
- Time Domain Features
- Other Features

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Statistical Features

- Mean: $\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$, where N is the number of samples and x_i is the i-th sample.
- Median: The middle value of the sorted samples.
- Mode: The most frequently occurring value in the samples.
- Standard Deviation: $\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$, where μ is the mean.
- Variance: $\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$, where μ is the mean.
- Skewness: $\gamma = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{x_i-\mu}{\sigma}\right)^3$, where σ is the standard deviation.
- Kurtosis: $\kappa = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{x_i-\mu}{\sigma}\right)^4 - 3$, where σ is the standard deviation.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Related library

```
# Import necessary libraries
import numpy as np
import pandas as pd
import scipy.stats as stats
import scipy.signal as signal
import pywt
import antropy as ant
import matplotlib.pyplot as plt
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Statistical features example

```
# Statistical Features
def extract_statistical_features(signal):
        mean = np.mean(signal)
        median = np.median(signal)
        std_dev = np.std(signal)
        variance = np.var(signal)
        skewness = stats.skew(signal)
        kurtosis = stats.kurtosis(signal)
        range_val = np.ptp(signal)
        features = {
        'Mean': mean,
        'Median': median,
        'Standard Deviation': std_dev,
        'Variance': variance,
        'Skewness': skewness,
        'Kurtosis': kurtosis,
        'Range': range_val
}
        return features
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Statistical features usage

```
# Extract statistical features
stat_features = extract_statistical_features(ecg_signal)
print("Statistical features:", stat_features)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Texture Features

- Contrast: $\sum_{i,j}(i-j)^2 P(i,j)$, where $P(i,j)$ is the co-occurrence matrix.
- Correlation: $\sum_{i,j} \frac{(i-\mu_i)(j-\mu_j)P(i,j)}{\sigma_i \sigma_j}$, where μ_i and μ_j are the means and σ_i and σ_j are the standard deviations of i and j.
- Energy: $\sum_{i,j} P(i,j)^2$, where $P(i,j)$ is the co-occurrence matrix.
- Homogeneity: $\sum_{i,j} \frac{P(i,j)}{1+|i-j|}$, where $P(i,j)$ is the co-occurrence matrix.
- Entropy: $-\sum_{i,j} P(i,j) \log P(i,j)$, where $P(i,j)$ is the co-occurrence matrix.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Related library

```
# Related library
from skimage.feature import greycomatrix, greycoprops
from skimage.measure import shannon_entropy
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Texture features examples

```
# Texture Features
def extract_texture_features(signal):
# Convert signal to 2D array for texture analysis
        signal_2d = np.reshape(signal, (int(np.sqrt(len(signal))), -1))

# Compute grey-level co-occurrence matrix (GLCM)
        glcm = greycomatrix(signal_2d, distances=[1], angles=[0], levels
             =256, symmetric=True, normed=True)
# Extract texture features
        contrast = greycoprops(glcm, 'contrast')[0,0]
        dissimilarity = greycoprops(glcm, 'dissimilarity')[0,0]
        homogeneity = greycoprops(glcm, 'homogeneity')[0, 0]
        energy = greycoprops(glcm, 'energy')[0, 0]
        correlation = greycoprops(glcm, 'correlation')[0, 0]
        asm = greycoprops(glcm, 'ASM')[0, 0]
        entropy = shannon_entropy(signal_2d)

        features = {
        'Contrast': contrast,
        'Dissimilarity': dissimilarity,
        'Homogeneity': homogeneity,
        'Energy': energy,
        'Correlation': correlation,
        'ASM': asm,
        'Entropy': entropy
}
        return features
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Texture features usage

```
texture_features = extract_texture_features(ecg_signal)
print("Texture features:", texture_features)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Entropy Features

- Shannon Entropy: $H(X) = -\sum_i p(x_i) \log p(x_i)$, where $p(x_i)$ is the probability of occurrence of x_i.
- Renyi Entropy: $H_\alpha(X) = \frac{1}{1-\alpha} \log \sum_i p(x_i)^\alpha$, where α is a parameter and $p(x_i)$ is the probability of occurrence of x_i.
- Tsallis Entropy: $H_q(X) = \frac{1}{q-1} (1 - \sum_i p(x_i)^q)$, where q is a parameter and $p(x_i)$ is the probability of occurrence of x_i.
- Differential Entropy: $h(X) = -\int p(x) \log p(x)\, dx$, where $p(x)$ is the probability density function.
- Cross Entropy: $H(p, q) = -\sum_i p(x_i) \log q(x_i)$, where $p(x_i)$ and $q(x_i)$ are the probability distributions.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Entropy features examples

```python

# Entropy Features
def extract_entropy_features(signal):
        sample_entropy = ant.sample_entropy(signal)
        spectral_entropy = ant.spectral_entropy(signal, sf=fs, method='welch')
        perm_entropy = ant.perm_entropy(signal, normalize=True)
        svd_entropy = ant.svd_entropy(signal, order=3, delay=1)
        app_entropy = ant.app_entropy(signal)
        lziv_complexity = ant.lziv_complexity(signal)

        features = {
                'Sample Entropy': sample_entropy,
                'Spectral Entropy': spectral_entropy,
                'Permutation Entropy': perm_entropy,
                'SVD Entropy': svd_entropy,
                'Approximate Entropy': app_entropy,
                'LZiv Complexity': lziv_complexity
                }

        return features
```


Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Entropy features usage

```
entropy_features = extract_entropy_features(ecg_signal)
print("Entropy features:", entropy_features)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Image Features

- Edges (using Sobel, Canny, etc.)
- Corners (using Harris, Shi-Tomasi, etc.)
- Blobs (using Laplacian of Gaussian, Difference of Gaussian, etc.)
- Maximum intensity
- Minimum intensity
- Mean intensity
- Standard deviation of intensity

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Image features examples

```
# Image Features
def extract_image_features(signal):
# Convert signal to 2D array for image analysis
        signal_2d = np.reshape(signal, (int(np.sqrt(len(signal))), -1))

# Extract image features
        edges = np.sum(np.abs(np.diff(signal_2d, axis=0))) + np.sum(np.
             abs(np.diff(signal_2d, axis=1)))
        corners = np.sum(signal_2d > np.mean(signal_2d) + 2 * np.std(
             signal_2d))
        blobs = np.sum(signal_2d > np.mean(signal_2d) + np.std(signal_2d
             ))
        mean_intensity = np.mean(signal_2d)
        std_intensity = np.std(signal_2d)
        max_intensity = np.max(signal_2d)
        min_intensity = np.min(signal_2d)

        features = {
                'Edges': edges,
                'Corners': corners,
                'Blobs': blobs,
                'Mean Intensity': mean_intensity,
                'Standard Deviation Intensity': std_intensity,
                'Max Intensity': max_intensity,
                'Min Intensity': min_intensity
        }

        return features
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Image features usage

```
image_features = extract_image_features(ecg_signal)
print("Image features:", image_features)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Frequency Domain Features

- Fourier Transform: $X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft}\, dt$, where $x(t)$ is the time-domain signal and $X(f)$ is the frequency-domain representation.
- Wavelet Transform: $W_\psi(a, b) = \int_{-\infty}^{\infty} x(t) \psi^* \left(\frac{t-b}{a}\right)\, dt$, where $x(t)$ is the signal, ψ is the wavelet function, a is the scale, and b is the translation.
- Power Spectral Density (PSD): Describes how the power of a signal is distributed with frequency.
- Cepstrum: $C(x) = \mathcal{F}^{-1}(\log |\mathcal{F}(x)|)$, where $\mathcal{F}$ denotes the Fourier transform.
- Mel-Frequency Cepstral Coefficients (MFCCs): Coefficients that represent the short-term power spectrum of a sound.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Frequency domain features examples

```
# Frequency Domain Features
def extract_frequency_domain_features(signal, fs):
        freqs, psd = signal.welch(signal, fs)
        dominant_freq = freqs[np.argmax(psd)]
        total_power = np.sum(psd)
        band_power = np.sum(psd[(freqs >= 0.5) & (freqs <= 40)])
        mean_freq = np.mean(freqs)
        median_freq = np.median(freqs)
        peak_freq = freqs[np.argmax(psd)]
        freq_variance = np.var(freqs)

    features = {
        'Dominant Frequency': dominant_freq,
        'Total Power': total_power,
        'Band Power (0.5-40 Hz)': band_power,
        'Mean Frequency': mean_freq,
        'Median Frequency': median_freq,
        'Peak Frequency': peak_freq,
        'Frequency Variance': freq_variance
}
        return features
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Frequency domain features usage

```
freq_features = extract_frequency_domain_features(ecg_signal, fs)
print("Frequency domain features:", freq_features)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Time Domain Features

- Temporal Moments: Statistical moments calculated over time.
- Zero Crossing Rate: $ZCR = \frac{1}{N-1} \sum_{i=1}^{N-1} \mathbb{1}_{\{x_i x_{i+1} < 0\}}$, where N is the number of samples and $\mathbb{1}$ is the indicator function.
- Autocorrelation: $R(\tau) = \sum_t x(t)x(t+\tau)$, where $x(t)$ is the signal and τ is the lag.
- Root Mean Square (RMS): $RMS = \sqrt{\frac{1}{N} \sum_{i=1}^{N} x_i^2}$, where N is the number of samples and x_i is the i-th sample.
- Signal Energy: $E = \sum_{i=1}^{N} x_i^2$, where N is the number of samples and x_i is the i-th sample.

Time Domain features examples

```
# Time Domain Features
def extract_time_domain_features(signal):
        rms = np.sqrt(np.mean(signal**2))
        zero_crossings = ((signal[:-1] * signal[1:]) < 0).sum()
        autocorrelation = np.correlate(signal, signal, mode='full')[len(
             signal)-1]
        mean_abs_dev = np.mean(np.abs(signal - np.mean(signal)))
        max_val = np.max(signal)
        min_val = np.min(signal)
        signal_energy = np.sum(signal**2)

        features = {
        'RMS': rms,
        'Zero Crossings': zero_crossings,
        'Autocorrelation': autocorrelation,
        'Mean Absolute Deviation': mean_abs_dev,
        'Max Value': max_val,
        'Min Value': min_val,
        'Signal Energy': signal_energy
}
        return features
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Time domain features usage

```
time_features = extract_time_domain_features(ecg_signal)
print("Time domain features:", time_features)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

How Features Reduce Dimensionality

- Feature extraction transforms data into a lower-dimensional space.
- Reduces the number of input variables, making models simpler and faster.
- Helps in removing redundant and irrelevant data.
- Techniques include Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), and t-Distributed Stochastic Neighbor Embedding (t-SNE).

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Feature Concatenation

- Combines information from multiple sources.
- Enhances the model's ability to learn complex patterns.
- Useful in scenarios with heterogeneous data types.
- Consider two feature vectors: **X1** and **X2**.
- Concatenation results in a new feature vector: $\mathbf{X} = [\mathbf{X1}, \mathbf{X2}]$.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Feature concatenation example and usage

```
# Combine all features into a single DataFrame
def combine_features(stat_features, time_features, freq_feature,
    entropy_features):
combined_features_list = []
for i in range(len(stat_features)):
combined_features = {**stat_features[i], **time_features[i], **
    freq_features[i], **entropy_features[i], }
combined_features_list.append(combined_features)
return pd.DataFrame(combined_features_list)

combined_features_df = combine_features(stat_features, time_features,
    freq_features, entropy_features)
print("Combined Features DataFrame:")
print(combined_features_df)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Feature normalization example and usage

```
from sklearn.preprocessing import StandardScaler
# Create DataFrame
df = pd.DataFrame(combined_features_df)

# Initialize the StandardScaler
scaler = StandardScaler()

# Apply the scaler to normalize the data
normalized_data = scaler.fit_transform(df)

# Convert the result back into a DataFrame
normalized_df = pd.DataFrame(normalized_data, columns=
    combined_features_df.columns)
print(normalized_df)
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Feature Reduction: Importance

- **Feature reduction** refers to reducing the number of input variables in a dataset.
- Helps to improve model performance by removing irrelevant or redundant features.
- Reduces computation time and storage space.
- Can prevent overfitting in machine learning models.
- Makes it easier to visualize high-dimensional data.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Techniques for Feature Reduction

- **Principal Component Analysis (PCA)**: Linear transformation that reduces dimensionality by projecting data onto the directions of maximum variance.
- **Linear Discriminant Analysis (LDA)**: Maximizes the separation between classes while reducing dimensions.
- **t-Distributed Stochastic Neighbor Embedding (t-SNE)**: Non-linear technique used for the visualization of high-dimensional datasets.
- **Autoencoders**: Neural network-based technique to compress and reconstruct the data, often used for non-linear feature reduction.
- **Feature Selection Methods**: Such as Recursive Feature Elimination (RFE), SelectKBest, and others.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Principal Component Analysis (PCA) Overview

- PCA is a statistical method for reducing the number of dimensions (features) in a dataset.
- It transforms the original features into a new set of uncorrelated features (principal components).
- The components are ordered by the amount of variance they capture from the data.
- PCA seeks to find the directions of maximum variance in the data and project the data along those directions.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

PCA: Algorithm and Mathematical Explanation

- Step 1: Standardize the Data Standardize the dataset to have a mean of 0 and a variance of 1 for each feature.

$$X_{std} = \frac{X - \mu}{\sigma}$$

  where X is the dataset, μ_x is the mean, and σ is the standard deviation of each feature.

- Step 2: Compute the Covariance Matrix between two random variables X and Y. The covariance matrix shows how the features vary together.

$$C(X, Y) = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu_x)(y_i - \mu_y)$$

  where:

  - N is the number of observations.
  - X_i and Y_i are the individual observations of the random variables X and Y.
  - μ_x and μ_y is the mean of the observations of X and (Y)

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

PCA: Algorithm and Mathematical Explanation

- Step 3: Compute the Eigenvectors and Eigenvalues Eigenvectors (principal components) and their corresponding eigenvalues are computed from the covariance matrix.

$$Cv = \lambda v$$

  where v is the eigenvector and λ is the eigenvalue.
- Step 4: Sort Eigenvalues Sort the eigenvalues in descending order. The eigenvectors corresponding to the largest eigenvalues are the most important principal components.
- Step 5: Choose Principal Components Select the top k eigenvectors corresponding to the largest eigenvalues, forming a new feature space.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Physical Interpretation of PCA

- PCA can be interpreted as finding the "directions" in the data that capture the most variance.
- The first principal component represents the direction in the feature space where the data varies the most.
- Each subsequent component is orthogonal to the previous one, capturing the remaining variance.
- Geometrically, PCA projects the data points onto a lower-dimensional plane or line while retaining the most important relationships in the data.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Advantages and Disadvantages of PCA

Advantages:

- Reduces dimensionality, improving computational efficiency.
- Removes multicollinearity by creating uncorrelated features.
- Retains most of the information (variance) with fewer components.
- Helps in visualizing high-dimensional data (e.g., 2D or 3D projection).

Disadvantages:

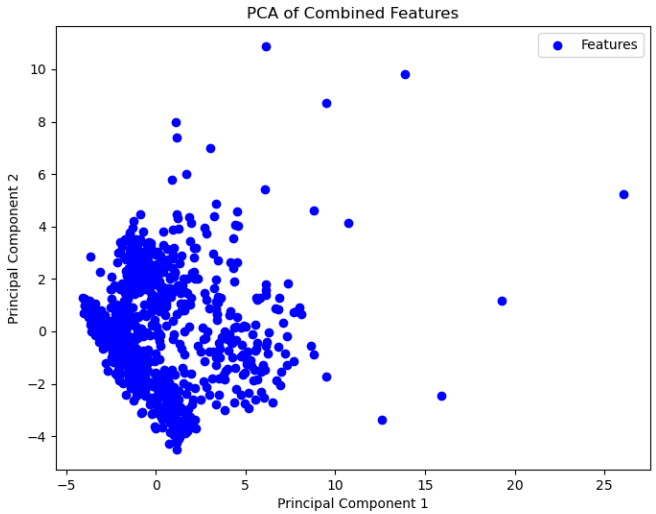
- PCA is sensitive to scaling of the data.
- The new features (principal components) may not be easily interpretable.
- Assumes linearity in the data, not suitable for non-linear relationships.
- Requires choosing the number of components to retain, which may not always be straightforward.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

PCA example and usage

```python
# Import necessary libraries
from sklearn.decomposition import PCA
import seaborn as sns

# Apply PCA
pca = PCA(n_components=2)
pca_result = pca.fit_transform(normalized_df)
```



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Kruskal Wallis example and usage

```
# Generate synthetic labels for two classes
# Perform Kruskal-Wallis test
kruskal_results = {}
for feature in normalized_df.columns:
        class_0 = normalized_df[feature][:400]
        class_1 = normalized_df[feature][401:]
        kruskal_results[feature] = stats.kruskal(class_0, class_1).
             pvalue
# Export Kruskal-Wallis test results to Excel
kruskal_results_df = pd.DataFrame(list(kruskal_results.items()), columns
     =['Feature', 'P-value'])
kruskal_results_df.to_excel('kruskal_wallis_results.xlsx', index=False)
print("Kruskal-Wallis test results have been exported to '
     kruskal_wallis_results.xlsx'.")

# Plot box plots for each feature
plt.figure(figsize=(12, 8))
features_to_plot = ['LZiv Complexity', 'Standard Deviation', 'Min Value'
     , 'Sample Entropy']
normalized_df['Class'] = Y
melted_df = pd.melt(normalized_df, id_vars='Class', var_name='Feature',
     value_name='Value')
melted_df = melted_df[melted_df['Feature'].isin(features_to_plot)]
sns.boxplot(x='Feature', y='Value', hue='Class', data=melted_df)
plt.xticks(rotation=90)
plt.title('Box Plot of Two Classes')
plt.show()
```


Classification pipeline

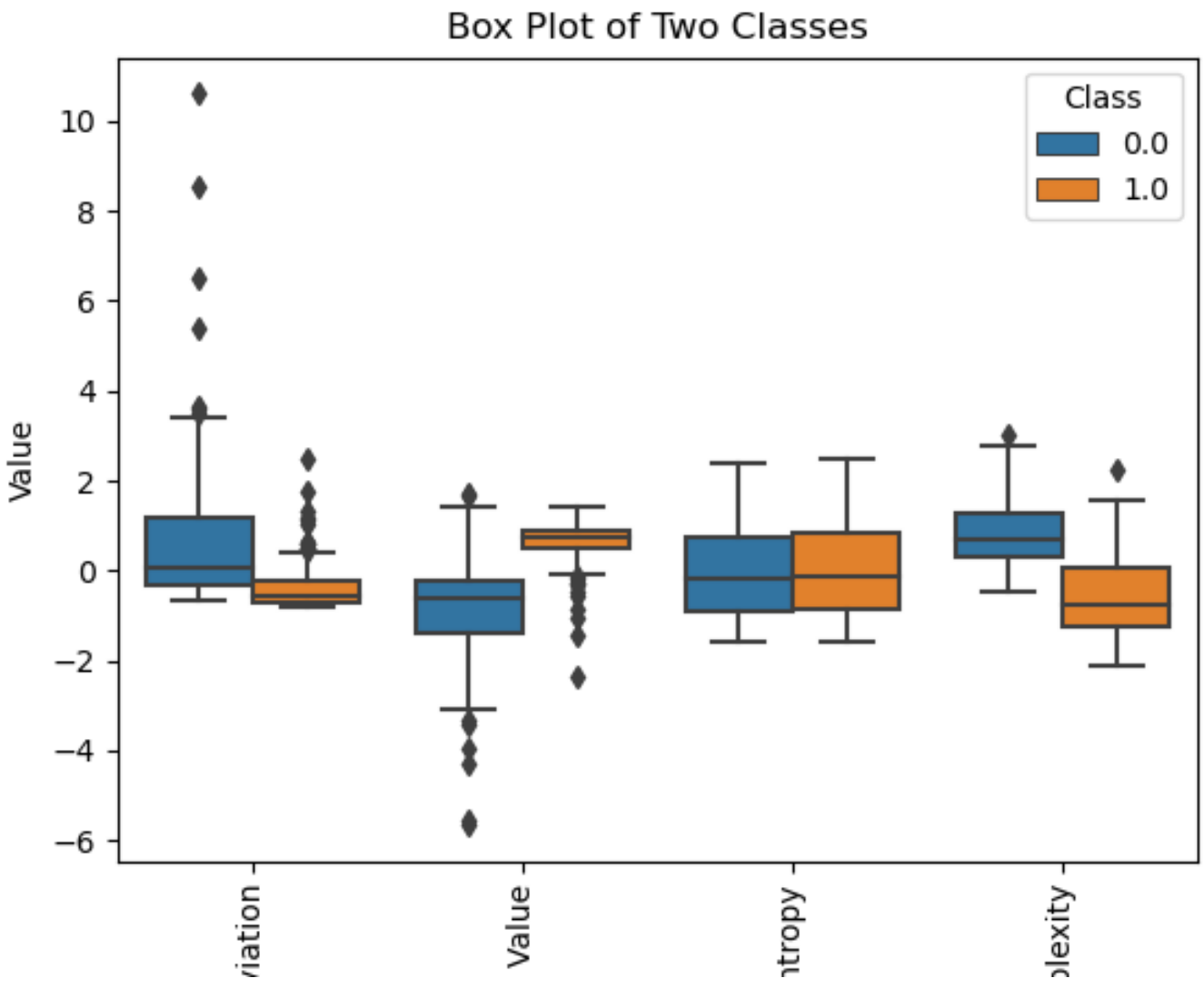
Data cleaning

Feature extraction

Feature reduction techniques

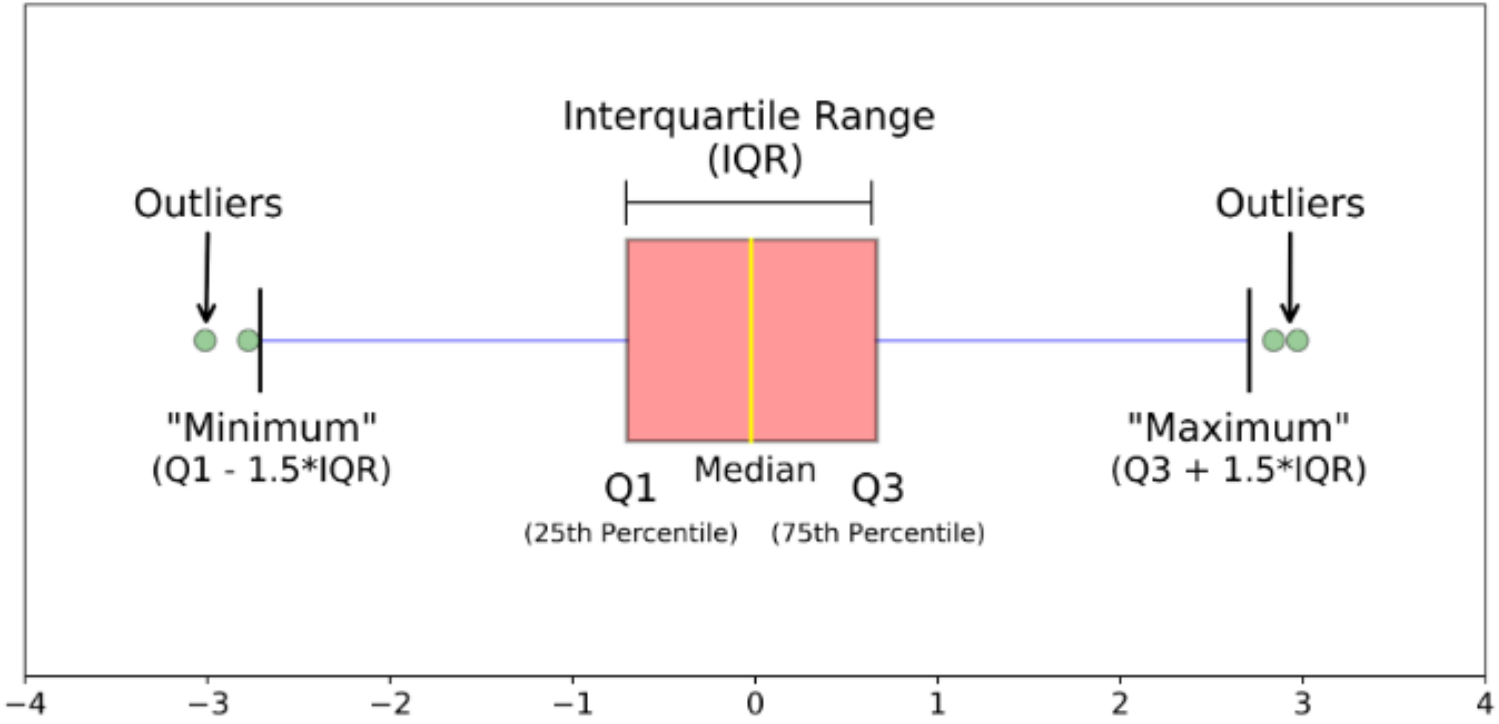
Support vector machine (SVM)

Kruskal Wallis example and usage



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Box plot explained



- **Median (Q2/50th percentile)**: The middle value of the data set.
- **First Quartile (Q1/25th percentile)**: The middle number between the smallest number (not the "minimum") and the median of the data set.
- **Third Quartile (Q3/75th percentile)**: The middle value between the median and the highest value (not the "maximum") of the dataset.
- **Interquartile Range (IQR)**: 25th to the 75th percentile.
- **Whiskers** (shown in blue)
- **Outliers** (shown as green circles)
- "**Minimum**": Q1 - 1.5*IQR
- "**Maximum**": Q3 + 1.5*IQR

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

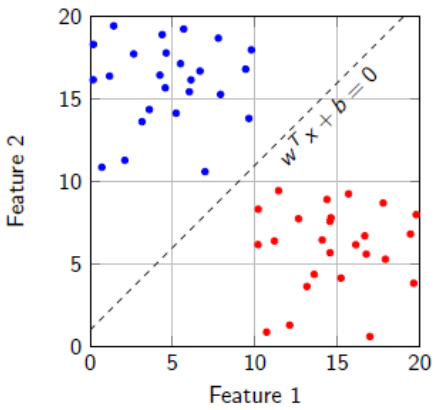
Introduction to SVM

- Support Vector Machines (SVM) are supervised learning models used for classification and regression tasks.
- SVMs aim to find the optimal hyperplane that maximizes the margin between different classes.
- They are effective in high-dimensional spaces and are versatile with different kernel functions.

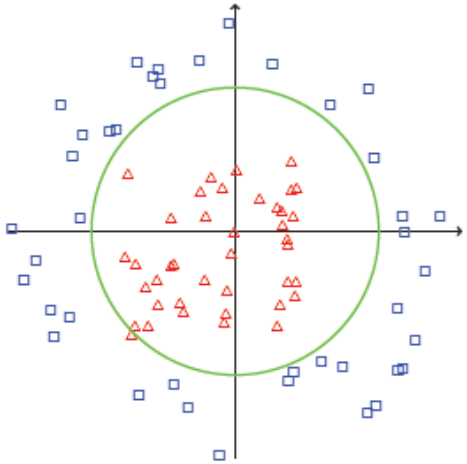
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Linear and Nonlinear Problems

- **Linear Problems**: Problems where data points can be separated by a straight line (2D) or a hyperplane (higher dimensions).
- **Nonlinear Problems**: Data points cannot be separated by a single straight line or hyperplane.
- **Kernel Trick**: SVM maps nonlinear problems into higher dimensions to make them linearly separable using kernels (e.g., polynomial, radial basis function (RBF)).



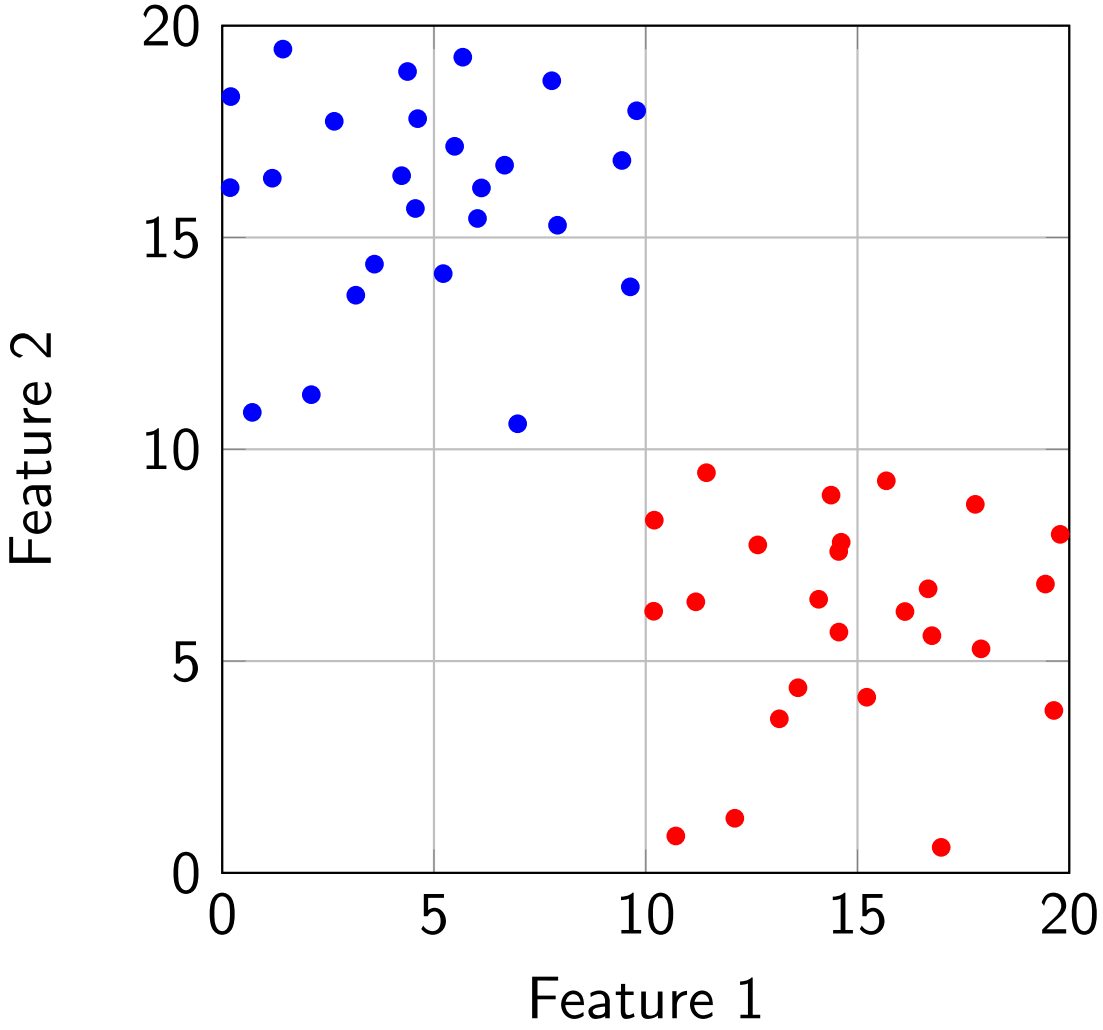
(a) Linear



(b) Nonlinear

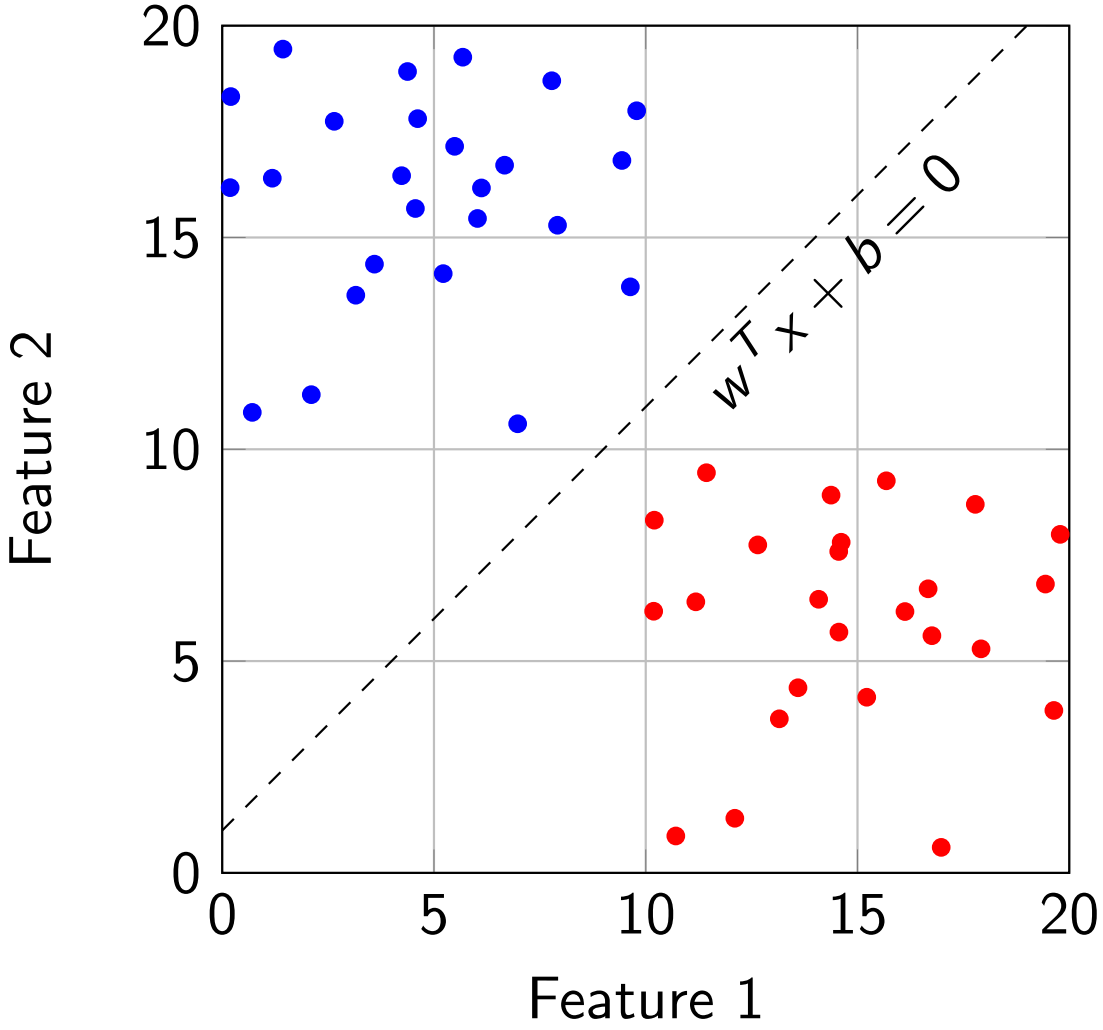
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Perceptron revisited: Linear separators



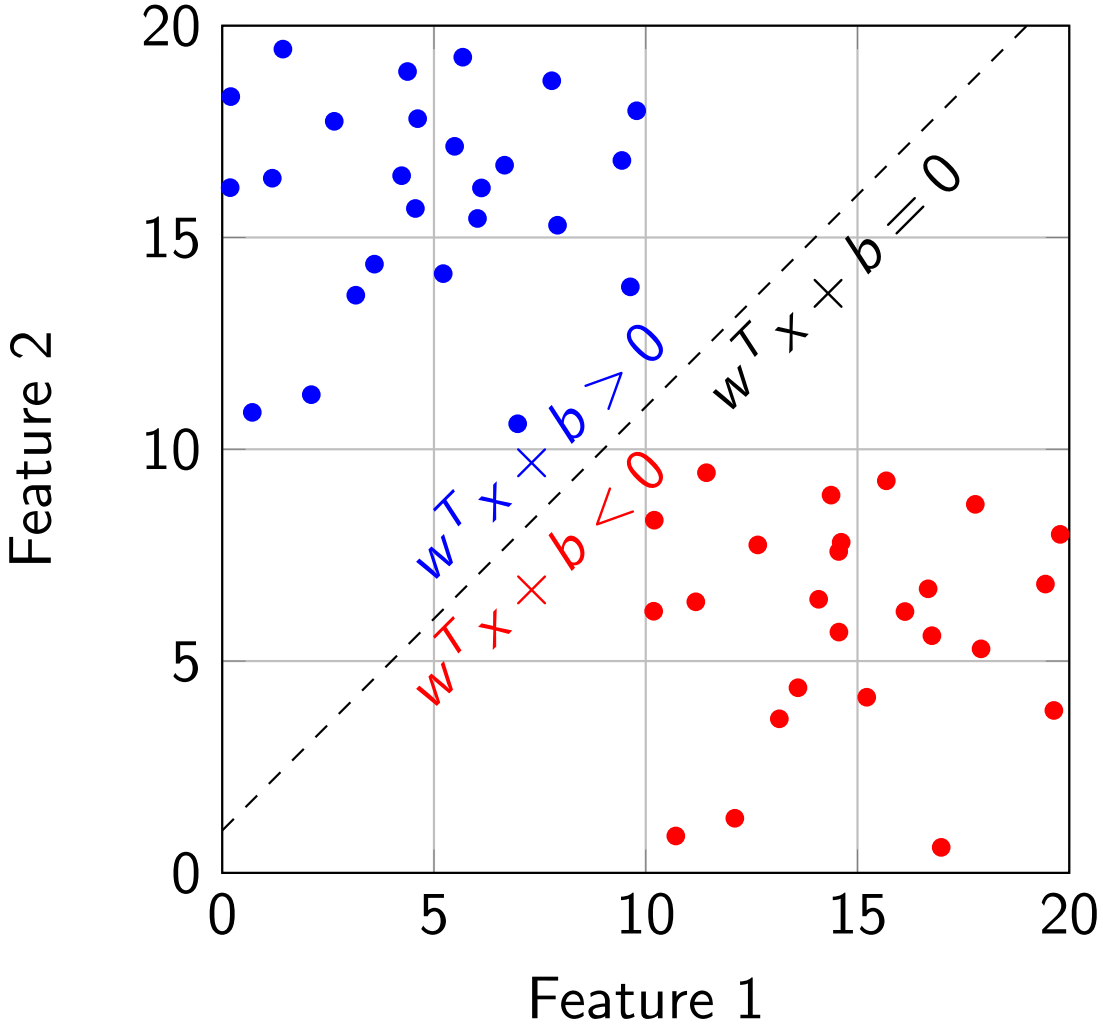
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Perceptron revisited: Linear separators



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

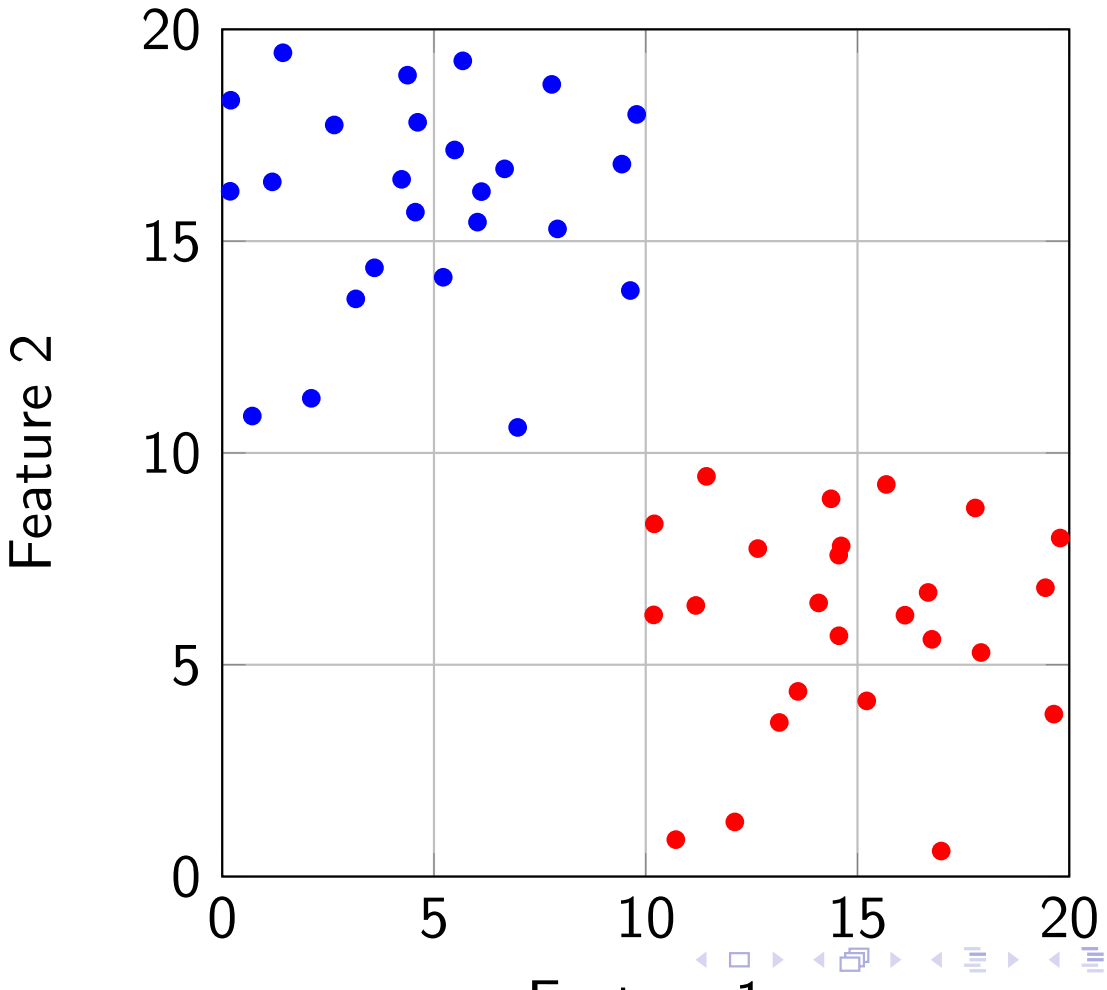
Perceptron revisited: Linear separators



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

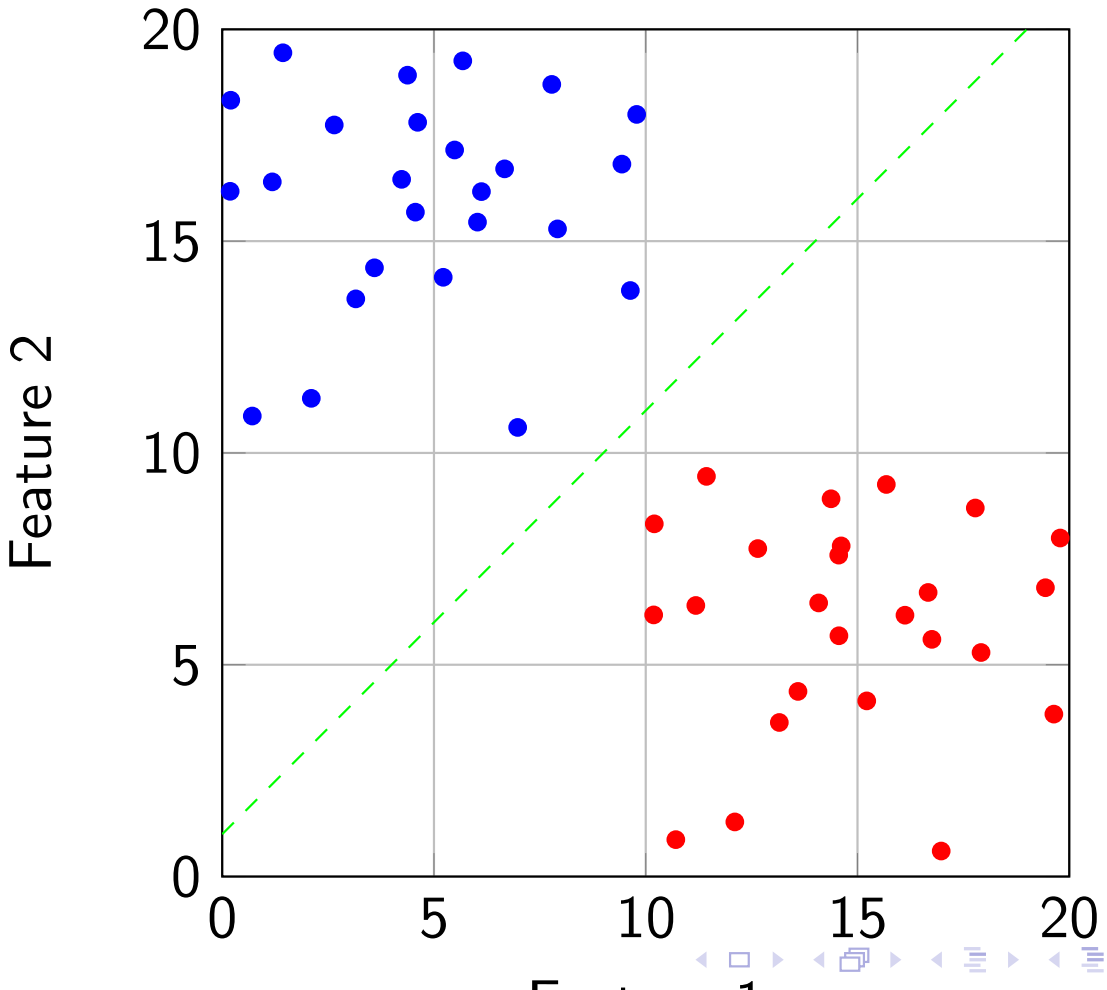
- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

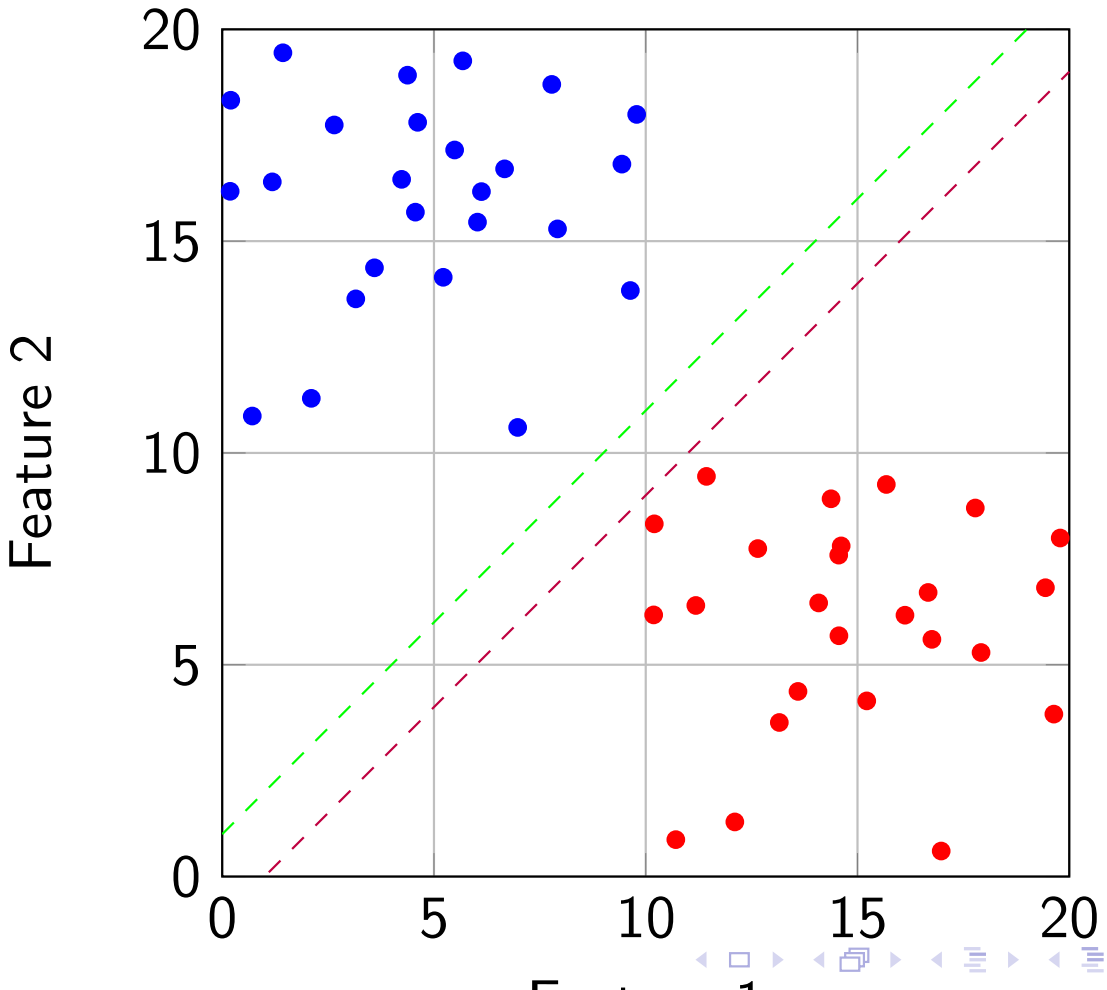
- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

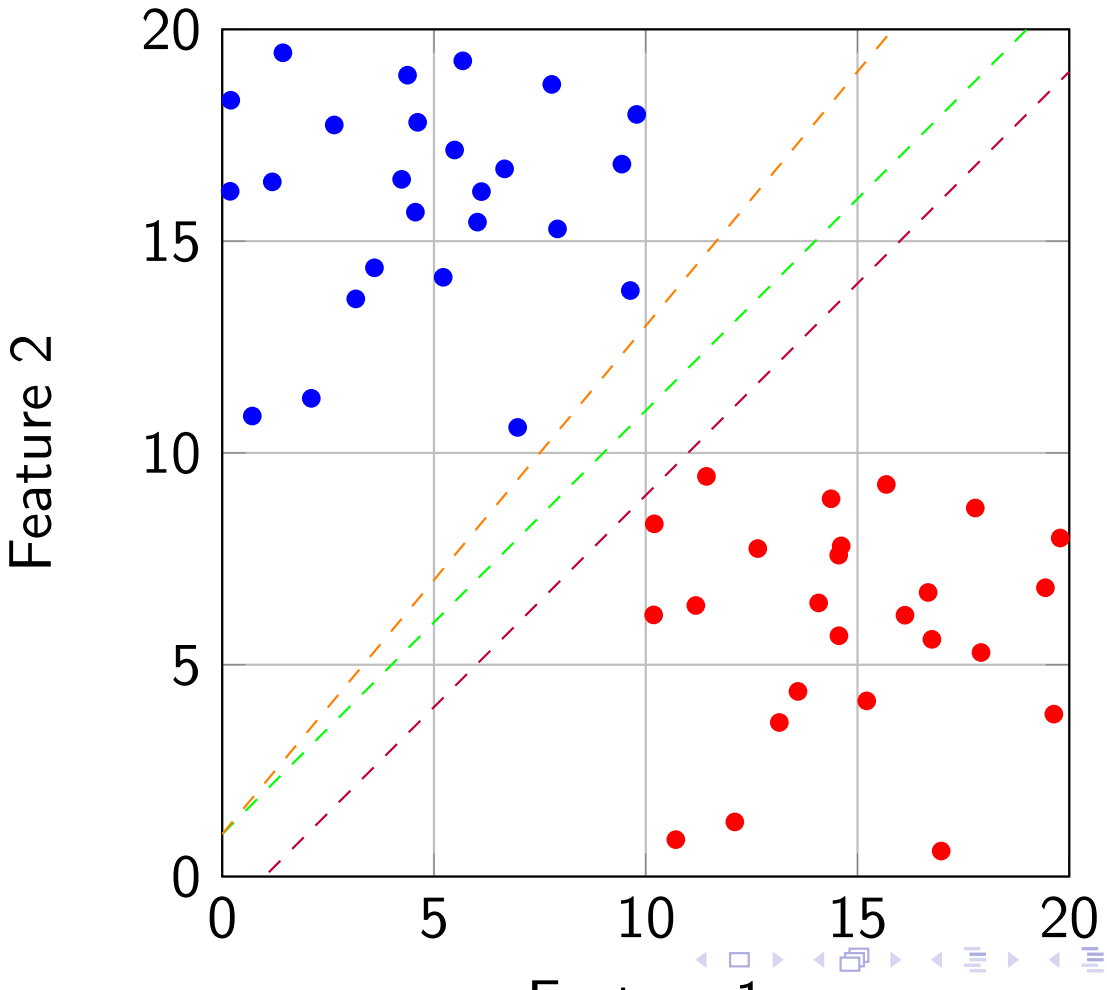
- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

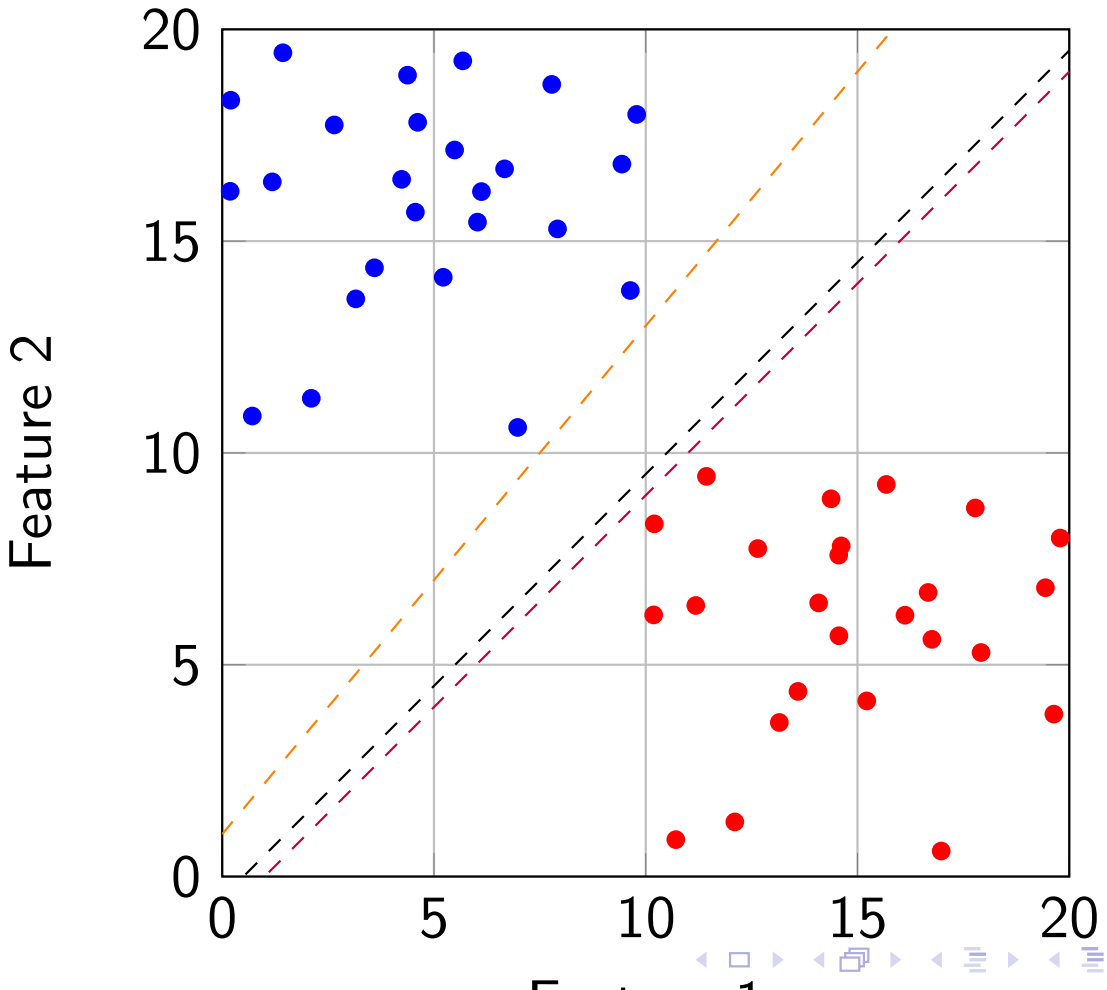
- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

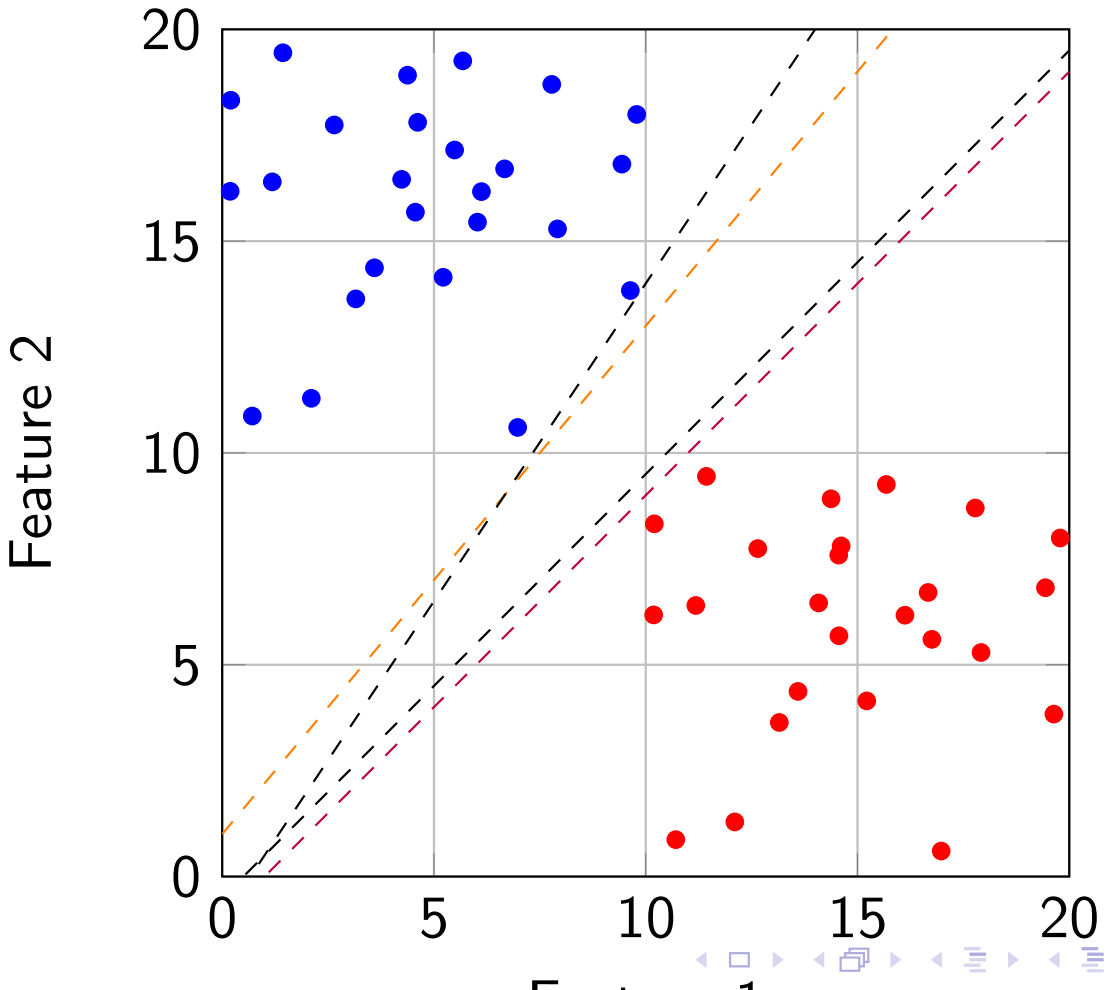
- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

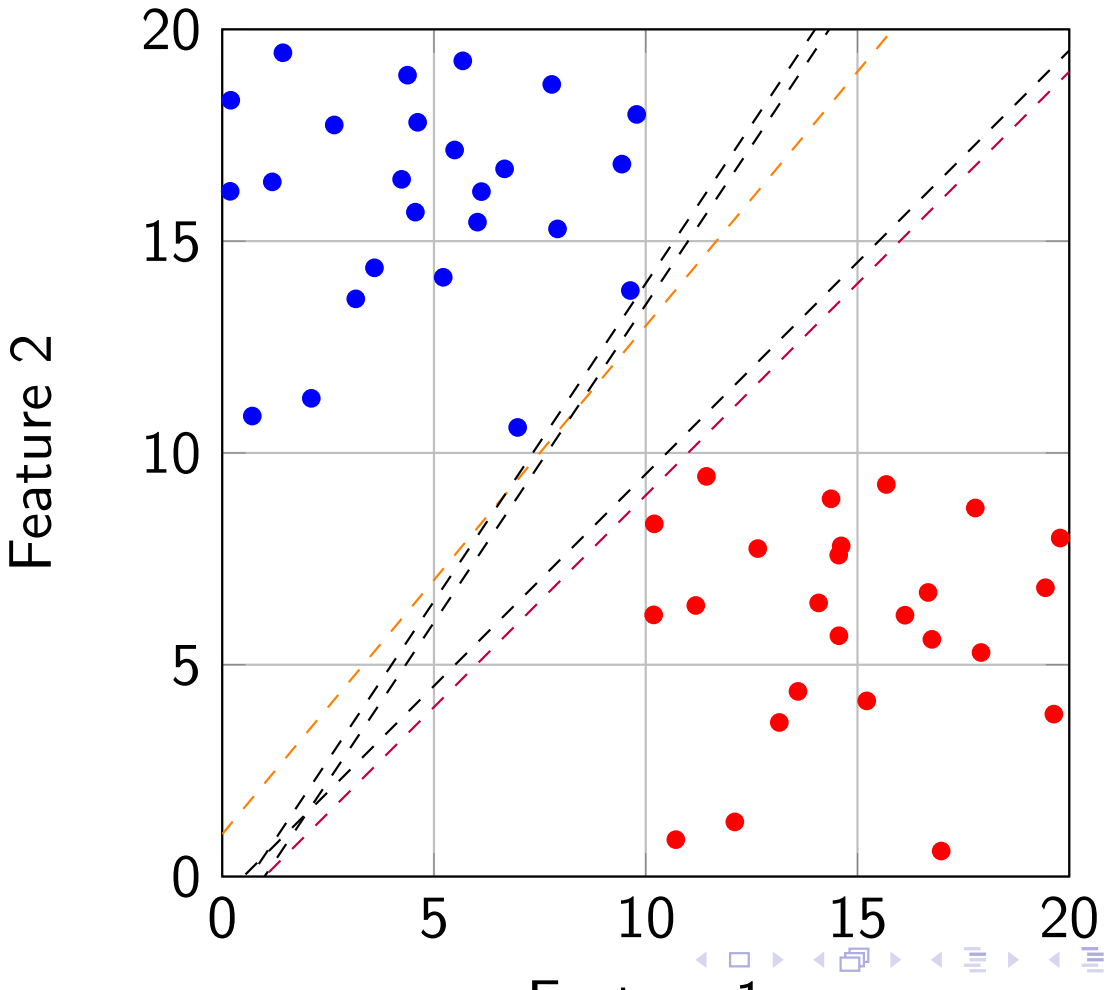
- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Multiple Decision Boundaries

- Multiple decision boundaries can exist when exploring different hyperplanes.
- Each boundary may correspond to a different separation strategy.



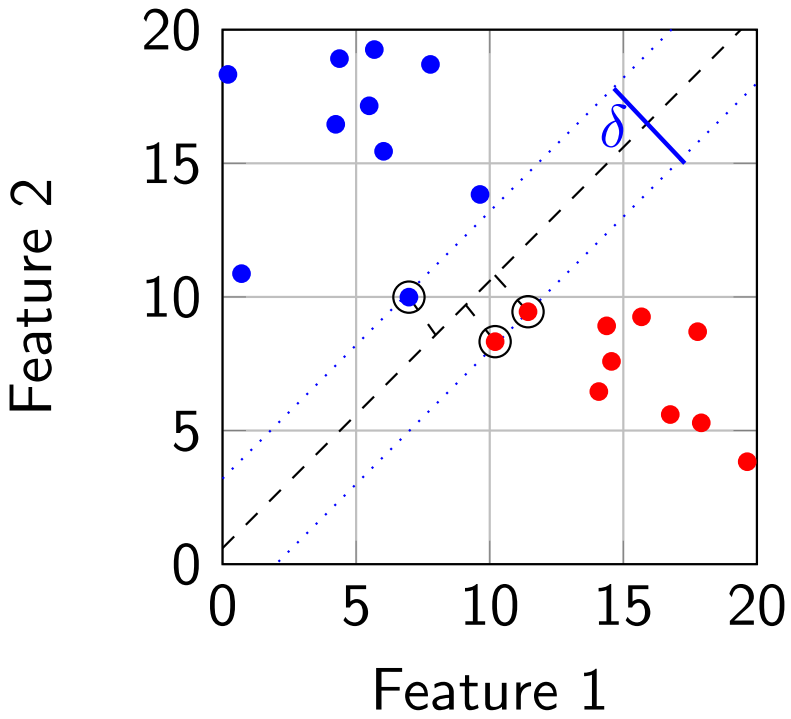
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Hyperplane and Classification Margin

- **Hyperplane**: A decision boundary that separates data points of different classes.
- **Classification Margin**: The distance between the hyperplane and the closest data points (support vectors).
- **Maximum Margin Classification**: SVM aims to maximize this margin for better generalization.
- A larger margin is preferred as it indicates better generalization to unseen data.
- The goal is to maximize the margin while correctly classifying the training data.

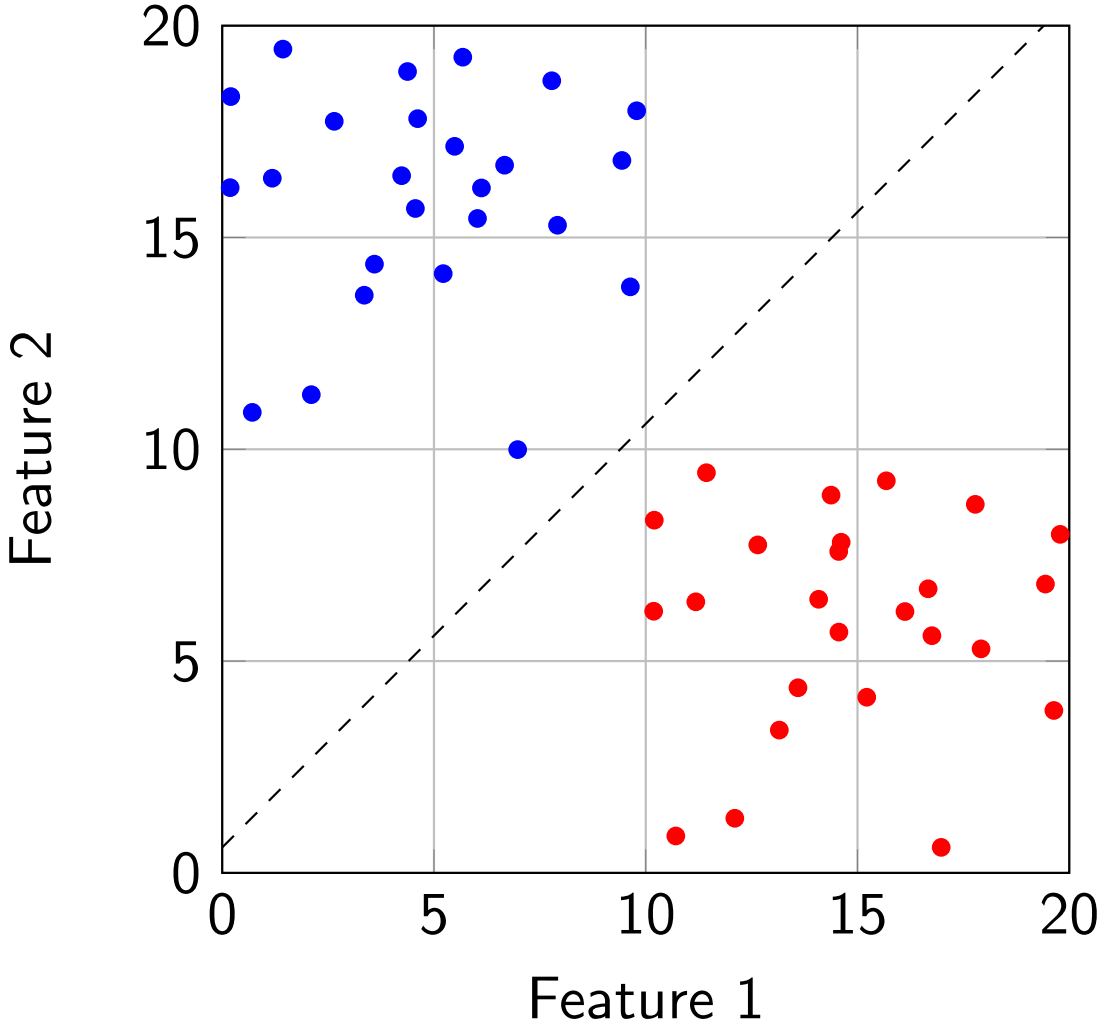
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Visualizing the Margin



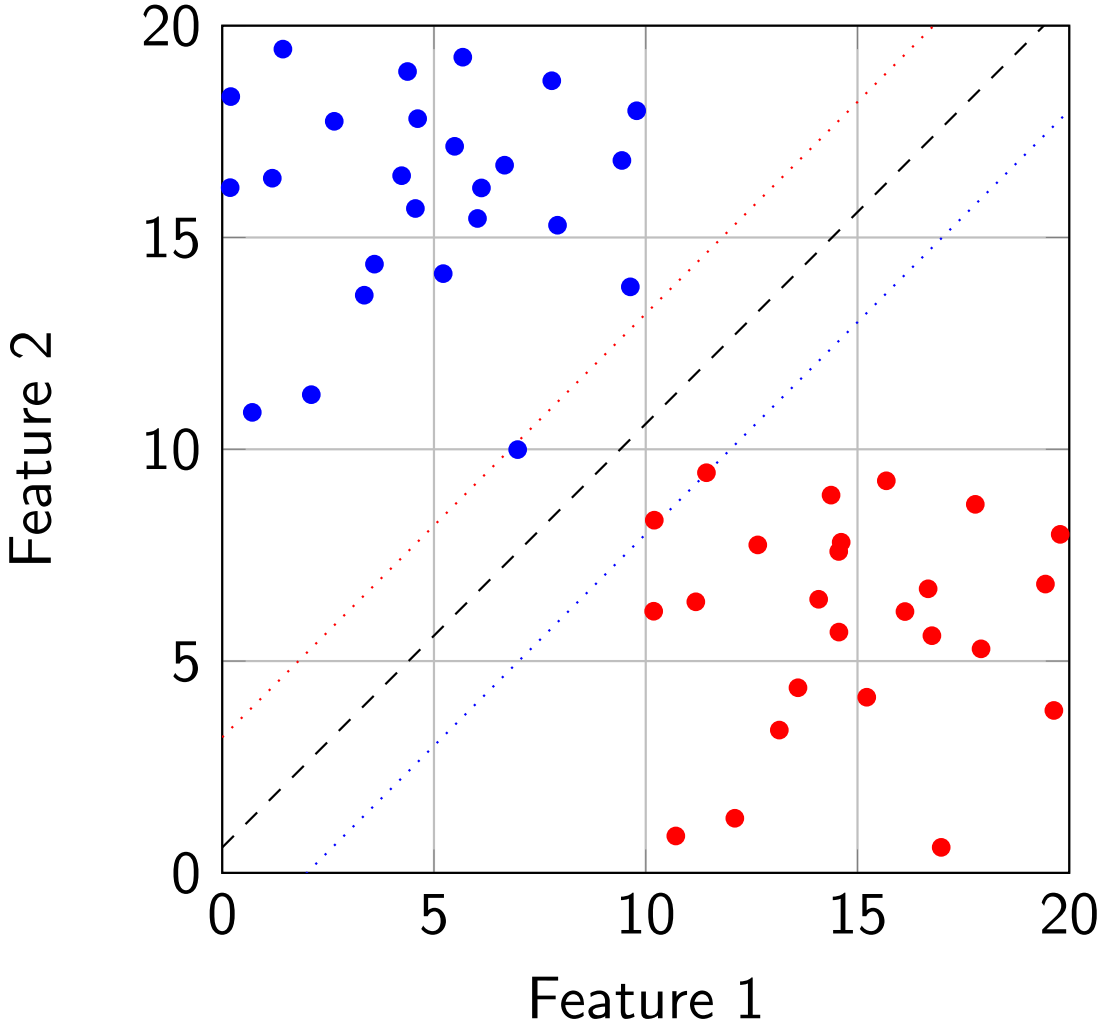
- **Hyperplane (Decision Boundary):** The black dashed line represents the decision boundary separating the two groups.
- **Margin (δ):** The region between the blue dotted lines. It is the distance between the hyperplane and the closest data points (support vectors).
- **Support Vectors:** Points that lie on the margin lines. For example, the points circled in black at coordinates.

Visualizing the Margin



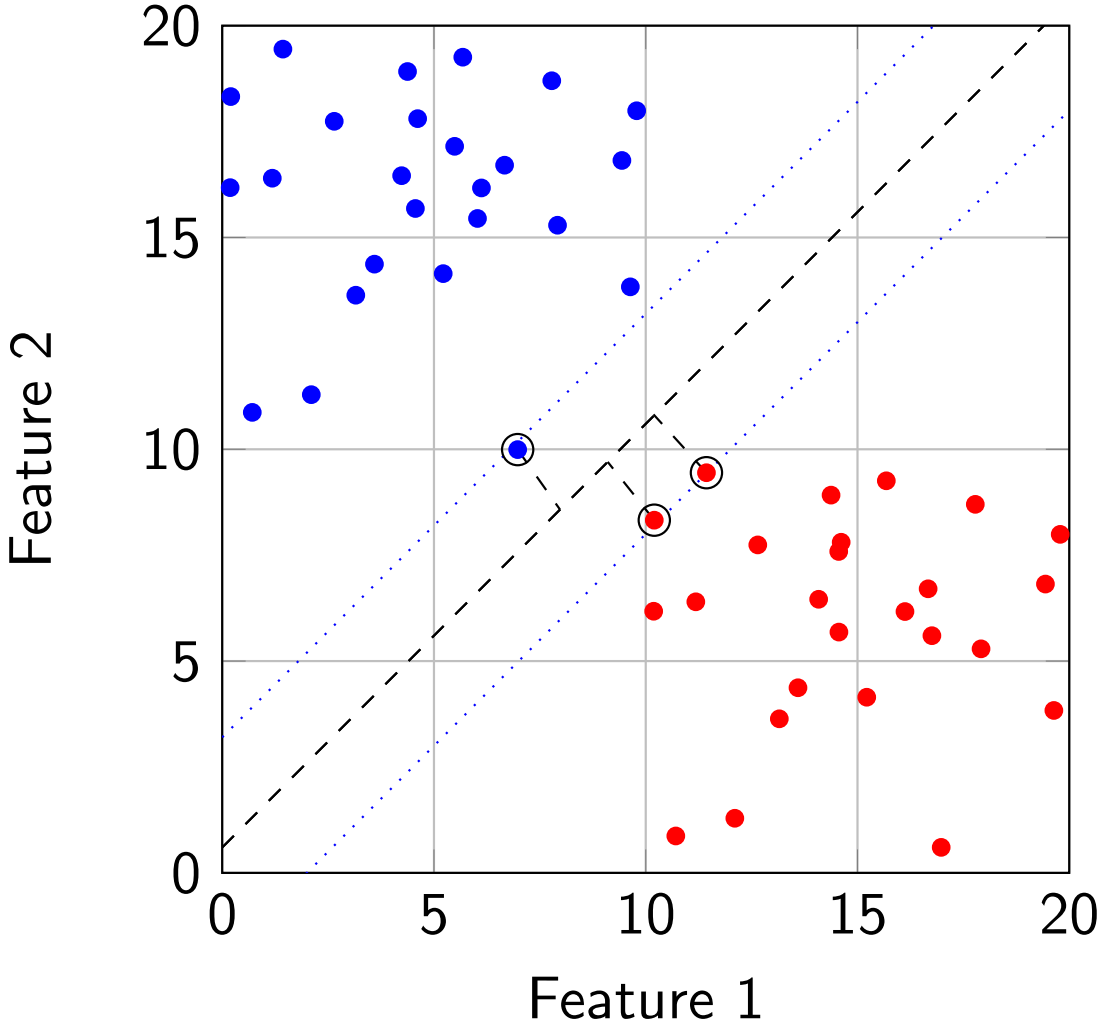
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Visualizing the Margin



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

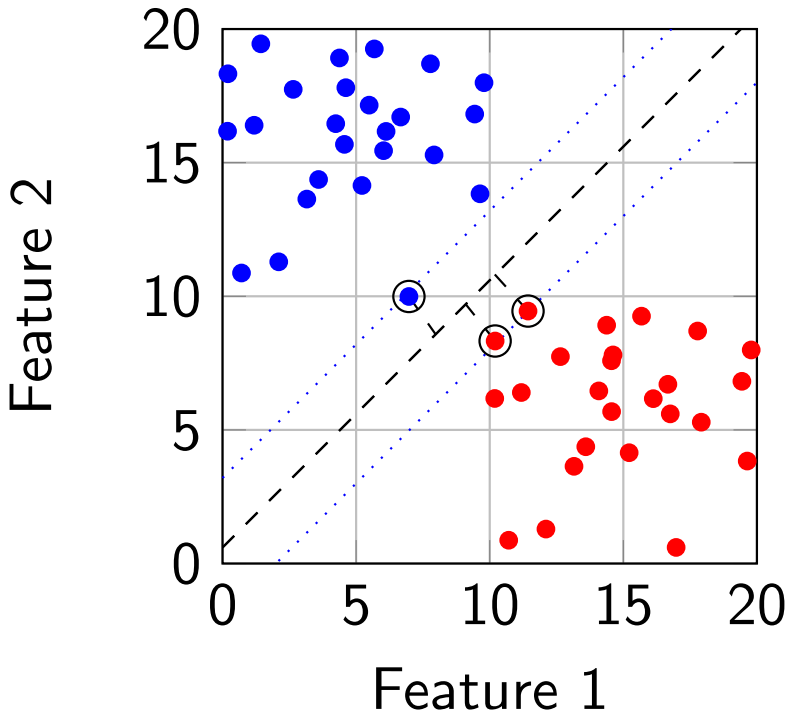
Visualizing the Margin



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

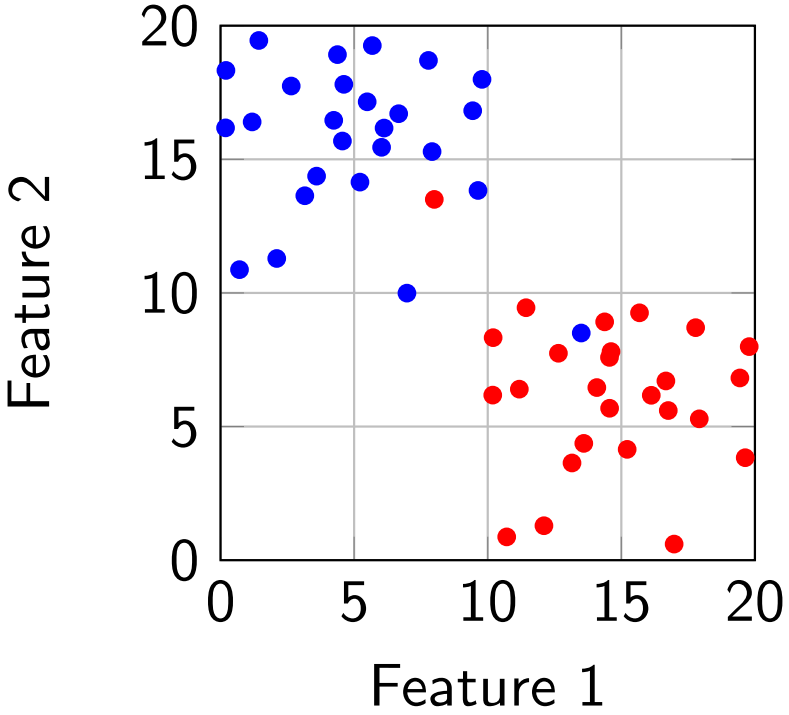
Soft Margin Classification

- **Hard Margin**: Assumes data is perfectly separable. Real-world data often contains noise or overlap between classes.



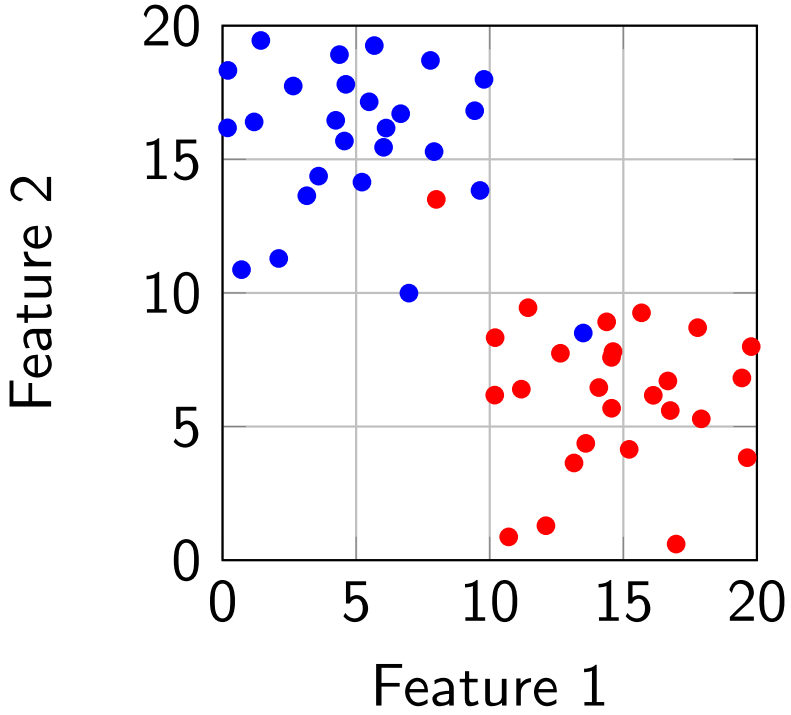
Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Soft Margin Classification



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Soft Margin Classification

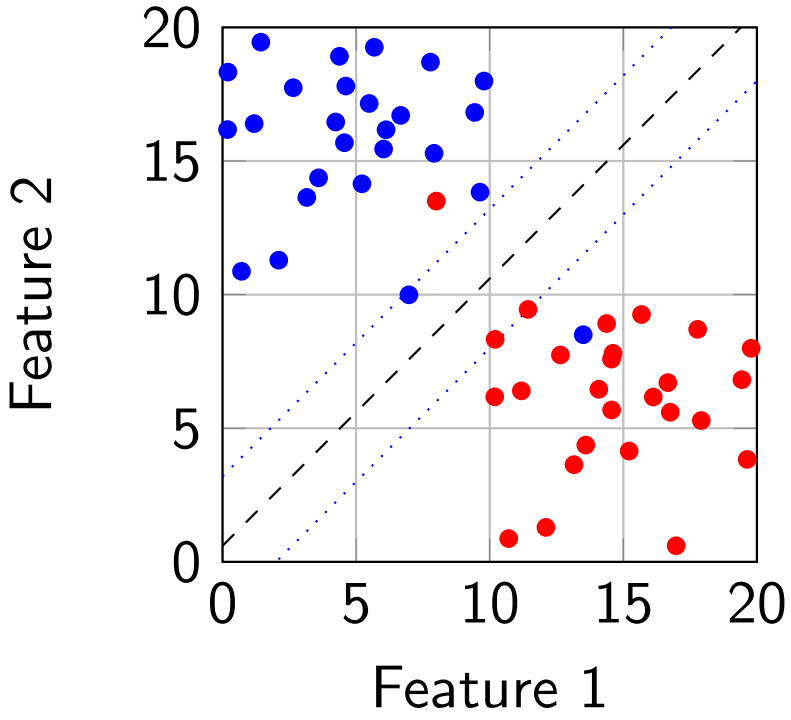


- What if the training set is not linearly separable?

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Soft Margin Classification

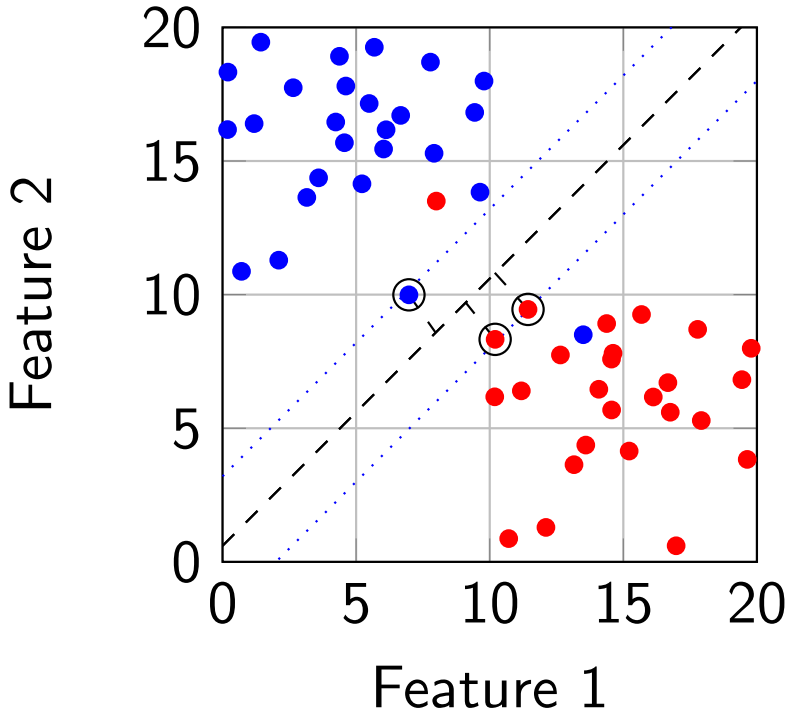
- **Soft Margin**: Allows for some misclassification to handle noisy or overlapping data.
- **Slack variables (ξ_i)** can be added to allow misclassification of difficult or noisy examples, resulting margin called soft.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Soft Margin Classification

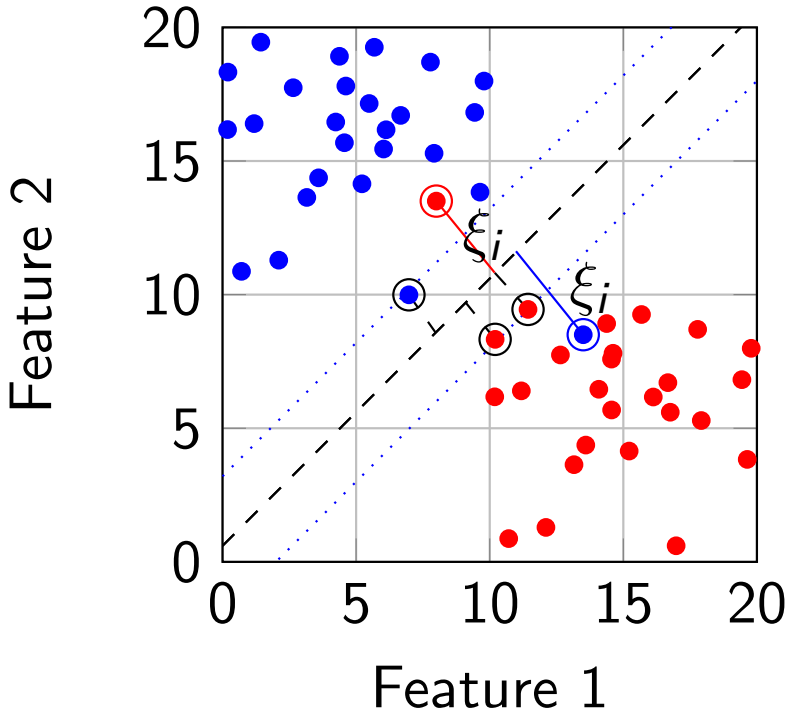
- **Soft Margin**: Allows for some misclassification to handle noisy or overlapping data.
- **Slack variables (ξ_i)** can be added to allow misclassification of difficult or noisy examples, resulting margin called soft.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Soft Margin Classification

- **Soft Margin**: Allows for some misclassification to handle noisy or overlapping data.
- **Slack variables (ξ_i)** can be added to allow misclassification of difficult or noisy examples, resulting margin called soft.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

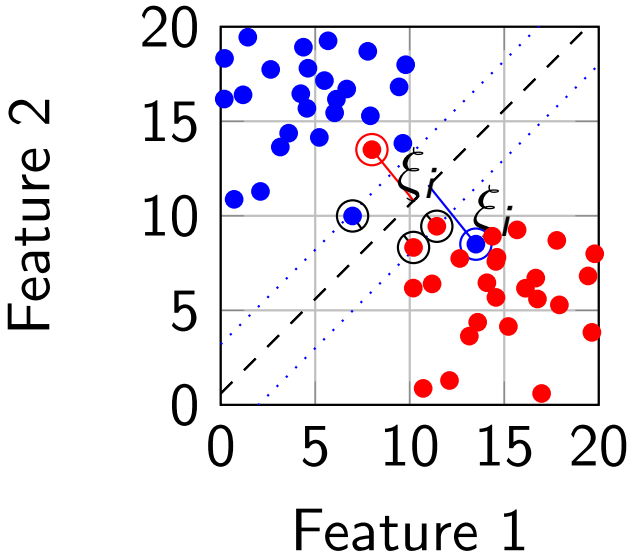
Slack Variables

- Slack variables ξ_i are introduced for each data point.
- $\xi_i \geq 0$ measures the degree of misclassification of the i-th data point.
- If $\xi_i = 0$, the point is correctly classified and lies outside the margin.
- If $0 < \xi_i \leq 1$, the point is correctly classified but within the margin.
- If $\xi_i > 1$, the point is misclassified.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Slack Variables

- Slack variables ξ_i are introduced for each data point.
- $\xi_i \geq 0$ measures the degree of misclassification of the i-th data point.
- If $\xi_i = 0$, the point is correctly classified and lies outside the margin.
- If $0 < \xi_i \leq 1$, the point is correctly classified but within the margin.
- If $\xi_i > 1$, the point is misclassified.



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Soft Margin SVM Optimization Problem

The optimization problem for Soft Margin SVM is:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

Subject to the constraints:

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \forall i$$

- w: weight vector defining the hyperplane.
- b: bias term.
- ξ_i: slack variable allowing misclassifications.
- C: is a regularization parameter that controls the trade-off between maximizing the margin and minimizing the classification error.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Linear SVMs: Overview

- The classifier is a separating hyperplane.
- Maximum margin classification ensures better generalization.
- Most "important" training points are support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points xi are support vectors with non-zero Lagrangian multipliers.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Non-Linear SVM

- Non-Linear SVM is used when data is not linearly separable in the original feature space.
- The data is transformed to a higher-dimensional feature space where it becomes linearly separable.
- This transformation is achieved using kernel functions.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Non-linear SVMs

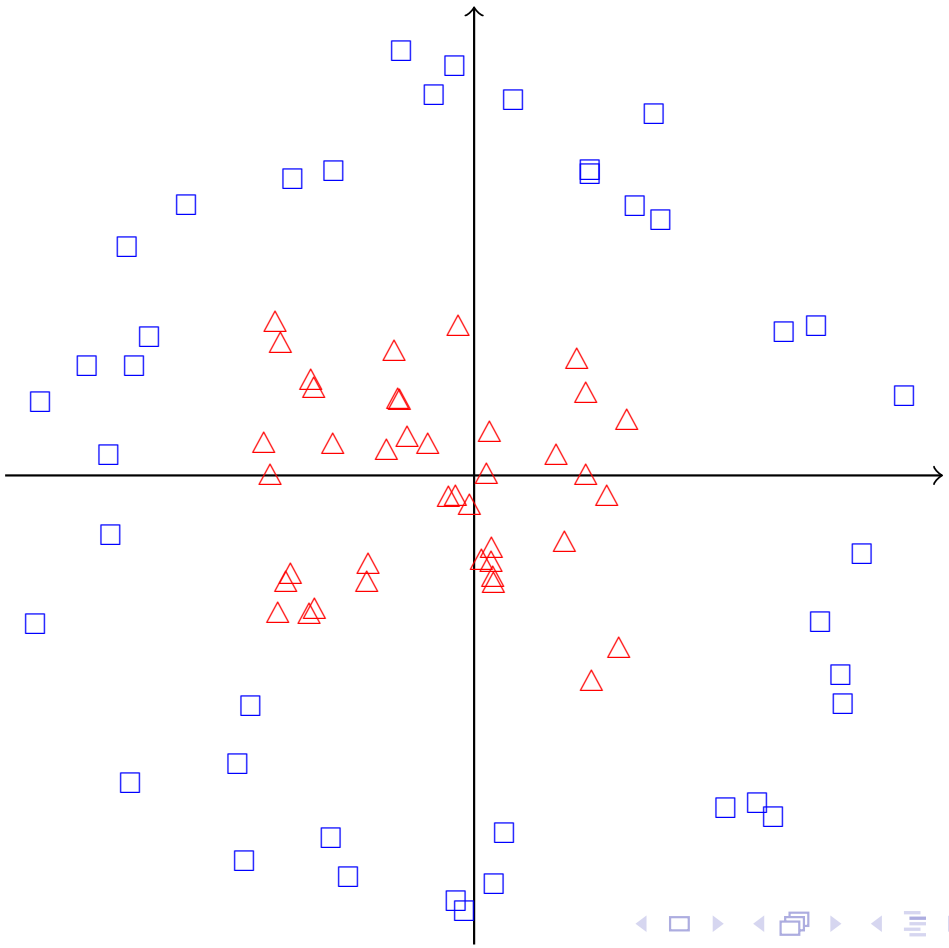
- Datasets that are linearly separable with some noise work out great.
- But what are we going to do if the dataset is just too hard?
- How about mapping data to a higher-dimensional space:
  - x_0
  - x_0
  - x_2
  - x_0

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Non-linear SVMs: Feature Spaces

- General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:

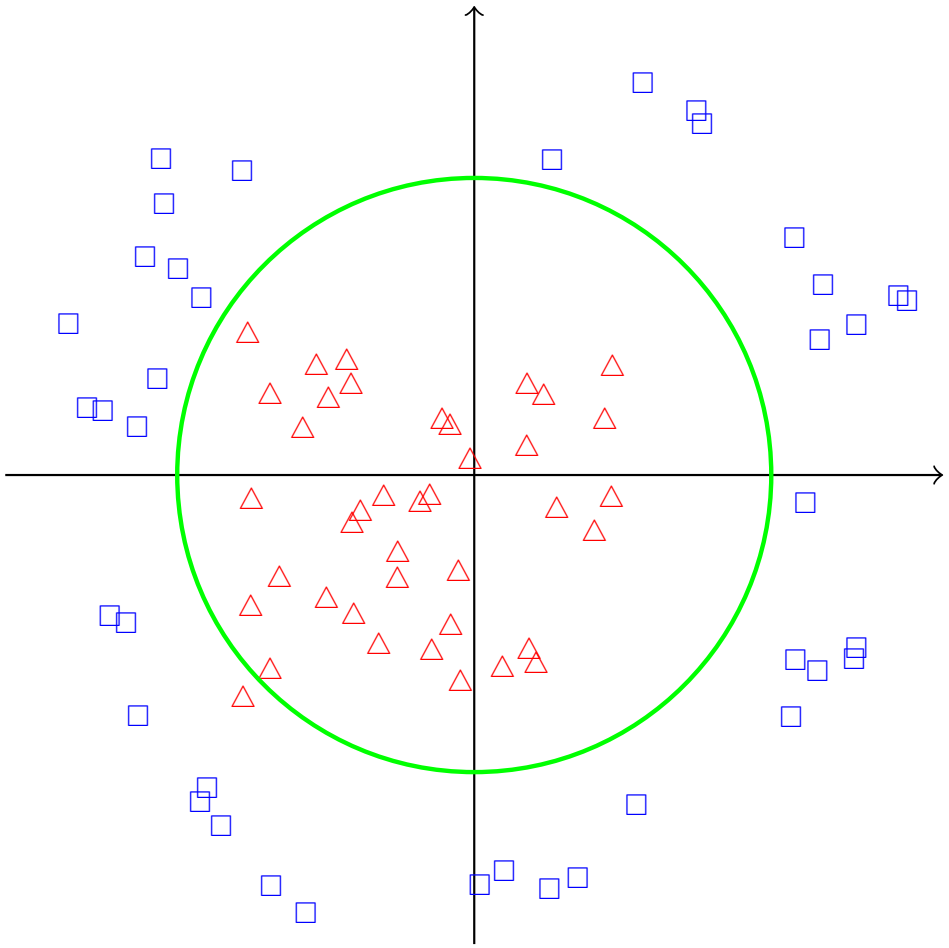
$$\Phi : x \to \phi(x)$$



Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Kernel Trick

$$\Phi : x \rightarrow \phi(x)$$



(a) Input space

Classification pipeline

Data cleaning

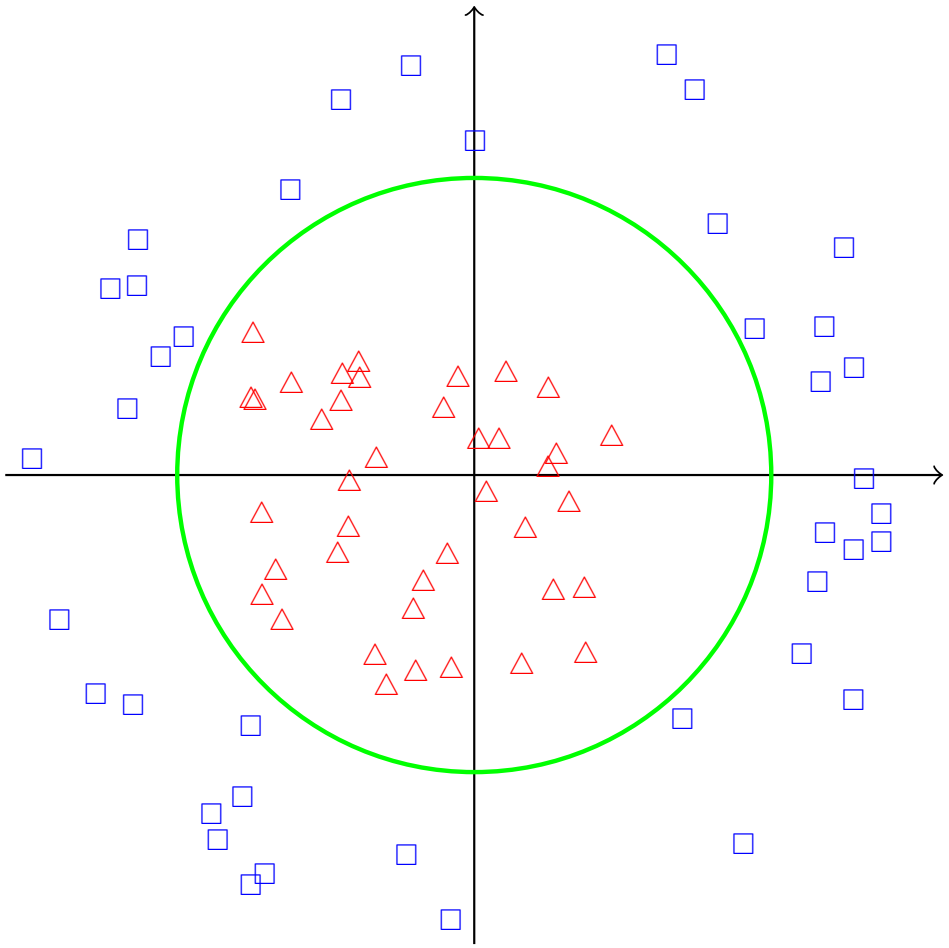
Feature extraction

Feature reduction techniques

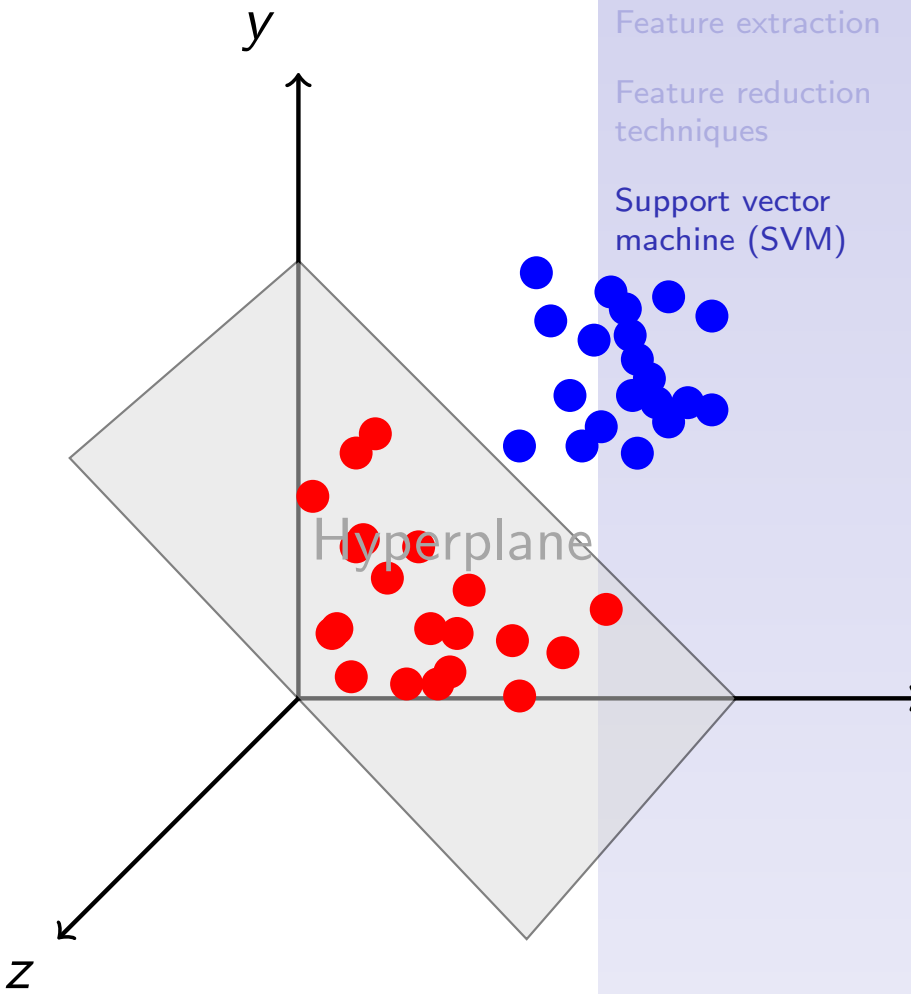
Support vector machine (SVM)

Kernel Trick

$$\Phi : x \rightarrow \phi(x)$$



(a) Input space



(b) Feature space

The Kernel Trick

- The linear classifier relies on inner product between vectors $K(x_i, x_j) = x_i^T x_j$.
- If every datapoint is mapped into high-dimensional space via some transformation $\Phi : x \to \phi(x)$, the inner product becomes:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

- A function that takes as its inputs vectors in the original space and returns the dot product of the vectors in the feature space is called a **kernel function**.
- More formally, if we have data $\mathbf{x}, \mathbf{z} \in X$ and a map $\phi : X \to \mathbb{R}^N$, then:

$$k(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

  is a **kernel function**.
- A kernel function is a function that is equivalent to an inner product in some feature space.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Optimization in SVM

- Objective: Maximize the margin while minimizing classification errors.
- Primal optimization problem:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$\text{subject to } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- Dual problem uses Lagrange multipliers for efficient computation.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Solving the Optimization Problem

- Quadratic programming (QP) methods are used to solve the dual problem.
- Dual formulation:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

$$\text{subject to } \sum_{i=1}^{n} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

- The solution provides the support vectors and decision boundary.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Examples of Kernel Functions

- Linear: $K(x_i, x_j) = x_i^T x_j$
- Polynomial of power p: $K(x_i, x_j) = (1 + x_i^T x_j)^p$
- Gaussian (radial-basis function): $K(x_i, x_j) = \exp(-\|x_i - x_j\|^2/2\sigma^2)$
- Mapping $\Phi : x \to \phi(x)$, where $\phi(x)$ is infinite-dimensional: every point is mapped to a function (a Gaussian); combination of functions for support vectors is the separator.
- Higher-dimensional space still has intrinsic dimensionality d (the mapping is not onto), but linear separators in it correspond to non-linear separators in original space.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Non-linear SVMs Mathematically

- Dual problem formulation: The solution is:

$$\max_{\alpha} Q(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

- Subject to:

$$\sum \alpha_i y_i = 0$$

$$\alpha_i \geq 0 \text{ for all } \alpha_i$$

- The optimization techniques for finding α_i's remain the same!
- The decision function is:

$$f(x) = \sum \alpha_i y_i K(x_i, x) + b$$

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Import SVM library and Implementation

```
# Import SVM and train test split
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
# Using 70% data for trainig and 30% data for testing
X_train, X_test, y_train, y_test = train_test_split(pca_result, Y,
     test_size=0.2, random_state=42)

# Initialize linear and nonlinear SVM models
linear_svm = SVC(kernel='linear')
nonlinear_svm = SVC(kernel='rbf')

# Fit the models with the selected features
linear_svm.fit(X_train, y_train)
nonlinear_svm.fit(X_train, y_train)

# Evaluate the models
linear_score = linear_svm.score(X_test, y_test)
nonlinear_score = nonlinear_svm.score(X_test, y_test)
```

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Plot SVM boundary

```
def plot_decision_boundary(clf, X, y, name):
# Convert X to NumPy array if it's a DataFrame
        if hasattr(X, "values"):
        X = X.values

# Create a mesh grid
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
        np.linspace(y_min, y_max, 500))

# Predict on grid points
        Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)

# Plot contours
        plt.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 50)
              , cmap=plt.cm.coolwarm, alpha=0.8)
        plt.contour(xx, yy, Z, levels=[0], linewidths=2, colors='k')  #
              Decision boundary

# Scatter plot of data points
        plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm,
              edgecolors='k')
        plt.xlabel('Feature 1')
        plt.ylabel('Feature 2')
        plt.title(name)
        plt.savefig(name)
        plt.show()


plot_decision_boundary(linear_svm, X_train, y_train,'SVM_Linear')
```


Classification pipeline

Data cleaning

Feature extraction

Feature reduction techniques

Support vector machine (SVM)

Linear and Nonlinear SVM decision boundaries

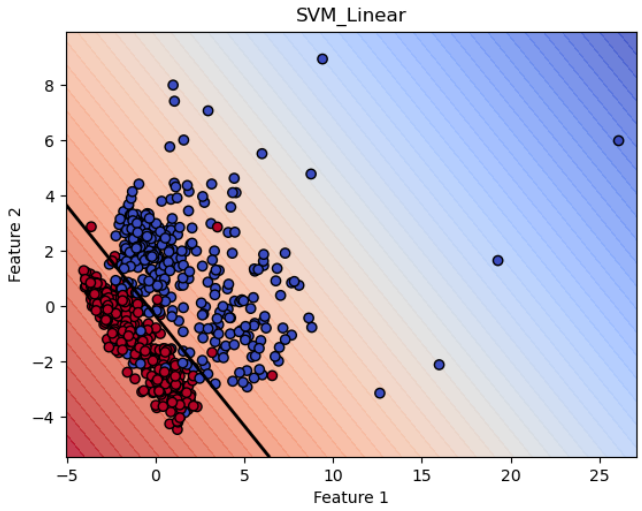


Figure: SVM linear classifier

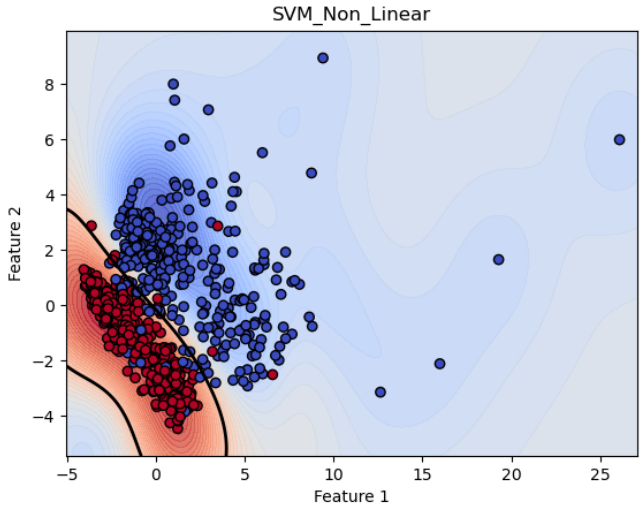


Figure: SVM nonlinear classifier (RBF kernel)

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)

Conclusion

- SVM is a powerful classifier for both linear and nonlinear problems.
- The choice of kernel function and the soft margin parameter C significantly impacts performance.
- Maximum margin classification ensures better generalization.

Classification pipeline
Data cleaning
Feature extraction
Feature reduction techniques
Support vector machine (SVM)